FINANCIAL ENGINEERING

KBP FFÚ & KSTP FIS

Case Studies in Capital Markets and Banking

Portfolio optimization and CAPM

Ing. Milan Ficura, Ph.D.
milan.ficura@vse.cz

EVROPSKÁ UNIE
Evropské strukturální a investiční fondy
Operační program Výzkum, vývoj a vzdělávání

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

Section 3:
Portfolio optimization and CAPM

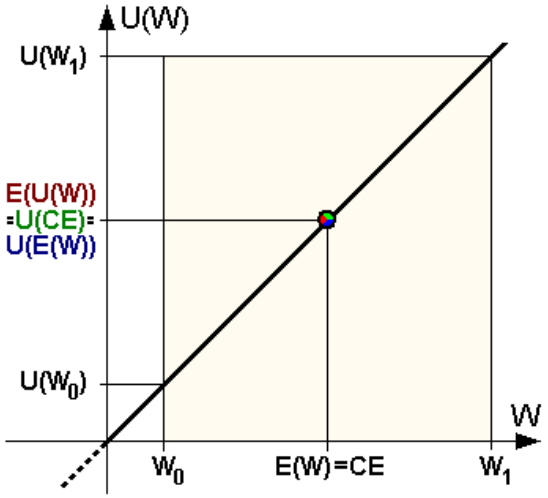
Portfolio optimization and CAPM

- **Modern Portfolio Theory (MPT)**
- Portfolio optimization in practice
- Case study 1 – MPT application on ETFs
- Case study 2A – MPT application on stocks
- Case study 2B – MPT application on stocks with CAPM

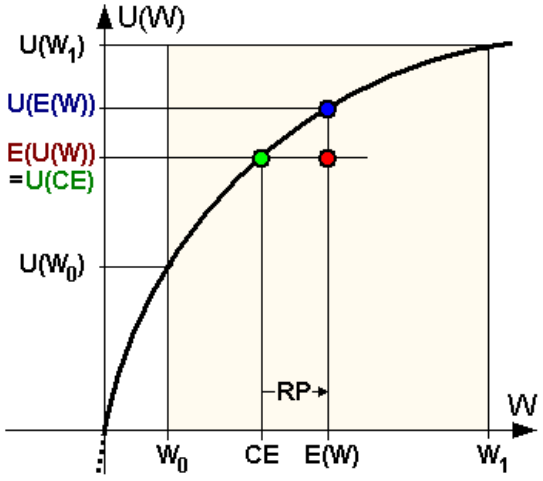
Risk-aversion

- Depending on investor preferences towards risk, we can distinguish:
  - **Risk-neutral investors** → $U[E(W)] = E[U(W)]$ **(ignore return variability)**
  - **Risk-averse investors** → $U[E(W)] \succ E[U(W)]$ **(prefer a certain payoff)**
  - **Risk-liking investors** → $U[E(W)] \prec E[U(W)]$ **(prefer a variable payoff)**
- **Utility functions** of the three types of investors are shown below

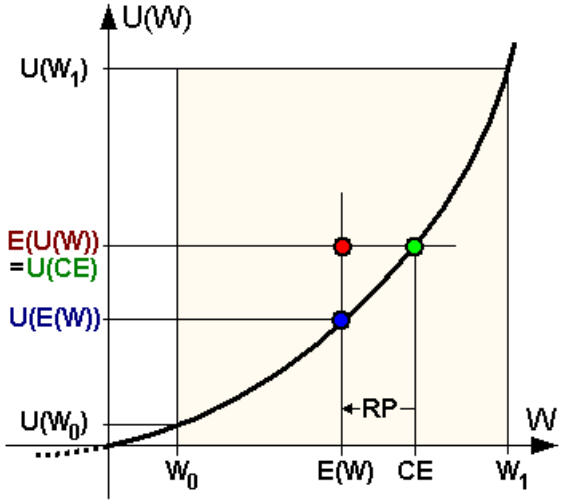
Risk-neutral:



Risk-averse:



Risk-liking:



- For the purposes of **asset pricing**, market participants will be viewed as **risk-averse** on average → demand positive risk premium

Asset pricing

- What is the **„fair“ asset price**?
  - The price at which **speculators** and **arbitrageurs** are at equilibrium
  - i.e. the price at which they are not motivated to move the prices further upwards or downwards with their purchases or sales
- The **equilibrium expected return** will consist of the:
  1. **Cost of funds** → The return that the investors demand for providing their capital (equal to the risk-free rate)
  2. **Market risk premium** → The additional return that the investors demand as a compensation for the variability of the assets returns (i.e. the possibility that it may provide lower return than expected)
  3. **Liquidity premium** → The additional return that the investors demand as a compensation for the illiquidity of the asset (i.e. the potential costs that may arise if the wanted to sell it before maturity)
- In the case of bonds there is also a **credit risk premium**, corresponding to the expected loss in the case of default (take not that this exists in addition to the **market risk premium**)

Risk & Return trade-off

- Assets with **higher variability of returns** tend to provide **higher expected returns** due to the **market risk premium**



Equity premium puzzle

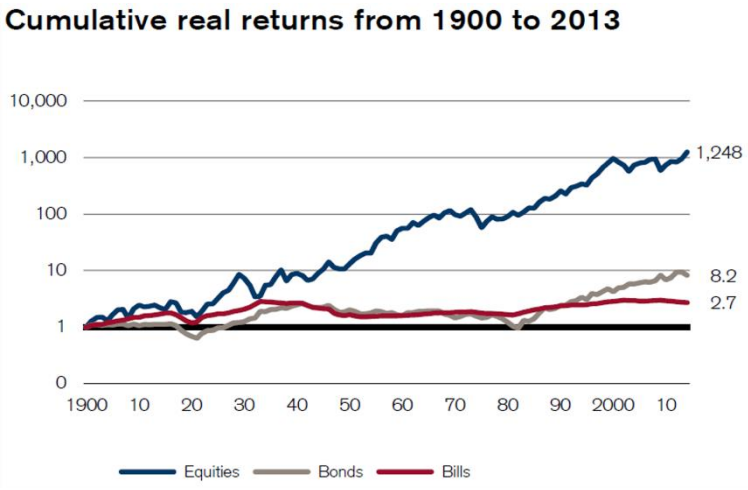
- There is actually an overly strong relationship between volatility and returns of different asset classes:
  - Stocks provide on average much higher returns than bonds
  - Corporate bonds provide higher returns than government bonds
  - Long-term government bonds outperform on average short-term bonds
- The **equity risk premium** of stocks vs. bonds is actually so high that it implies unrealistic degrees of risk-aversion of the bond investors (**equity premium puzzle**):

Premium of US equities vs. US bills and bonds:	**T-Bills**	**T-Bonds**
Ibbotson Associates (1926-2000):	6,3%	--
Dimson, Marsh, Staunton (1900-2001):	5,6%	4,8%
Dimson, Marsh, Staunton (1900-2012):	5,3%	4,3%

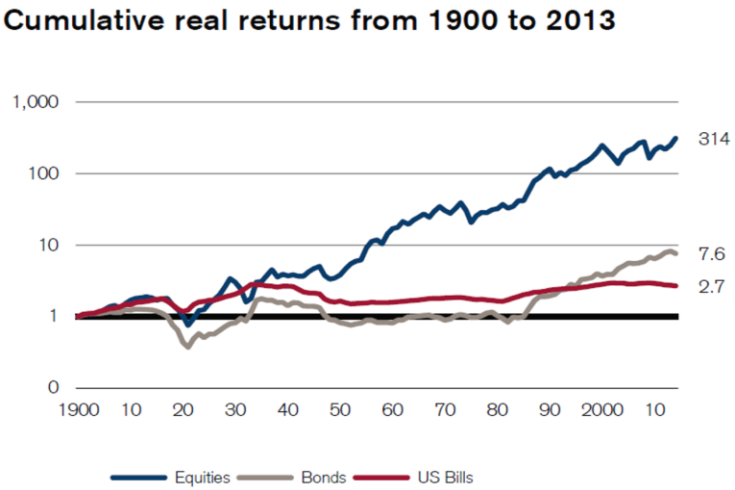
Premium of global capitalization weighted equity investment vs:	**Bills**	**Bonds**
Dimson, Marsh, Staunton (1900-2001):	4,6%	4,3%
Dimson, Marsh, Staunton (1900-2012):	4,1%	3,3%

Dimson, Marsh, Staunton (2013)

US cumulative real returns:



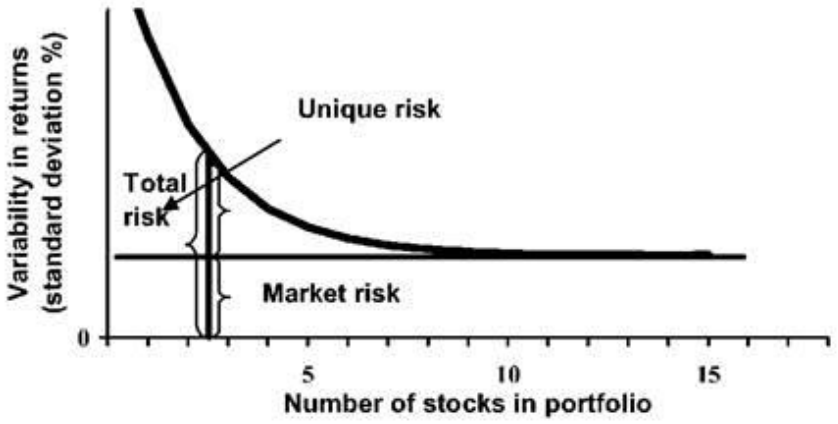
Global cumulative real returns:

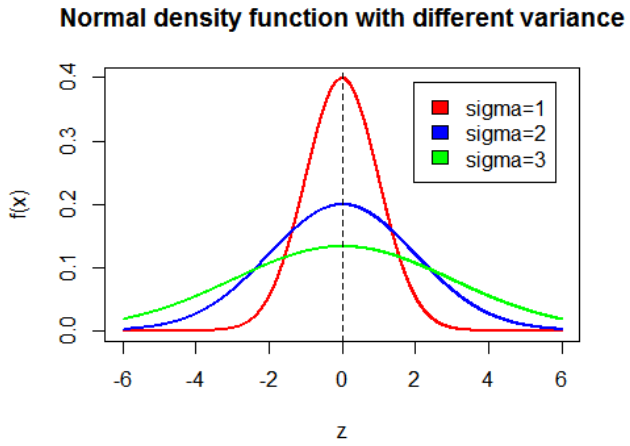


- There is no generally agreed explanation for the equity premium puzzle
- Among partial explanations is:
  - **Selection and survivorship bias** if the premium is computed only from the US stocks or few of the most developed markets
  - **Institutional factors** – Some investors may be incentivized to hold bonds
  - **Cognitive biases such as prospect theory** – Investory may be overly risk-averse towards possible losses

Diversification

- Do investors always require higher expected return as a compensation for higher variability of the asset? **No, because of effects of diversification**
- **Diversification example (uncorrelated assets)**:
  - Consider an equally-weighted portfolio of ***K* uncorrelated assets**, each with normaly distributed returns $N(\mu, \sigma)$
  - The portfolio return would then be equal to $N\left(\mu, \sigma/\sqrt{K}\right)$
  - i.e. if $\sigma = 10\%$, then for $K = 100$ assets the standard deviation of portfolio returns would be $\sigma_P = 1\%$ and for $K = 10\,000$ only $\sigma_P = 0.1\%$
  - The investor would be able to eliminate almost all risk with diversification
- Why do market risk premia exist then? Because the **asset returns are correlated → Market risk cannot be fully eliminated**



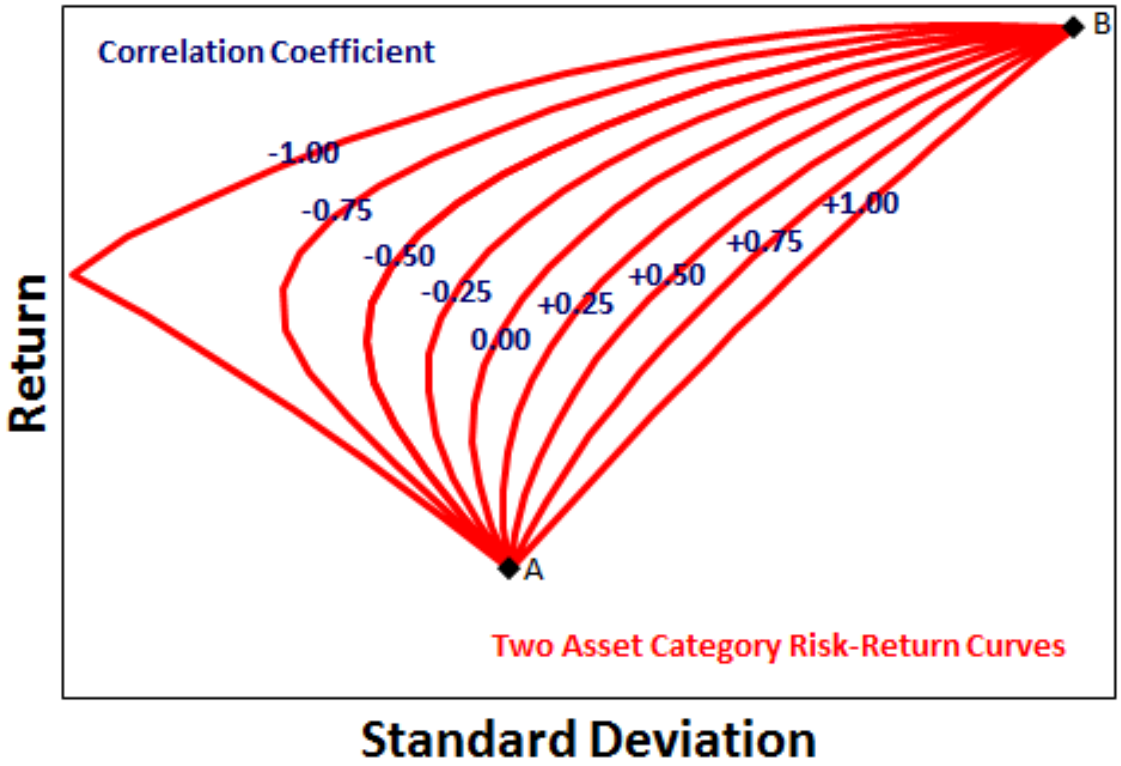


Modern Portfolio Theory (Markowitz, 1952)

- Assumes Gaussian distribution of returns
  - We can use σ as a measure of risk
- In the case of a **two-asset portfolio** of assets A and B with:
  - Expected returns: μ_A and μ_B
  - Expected volatilities: σ_A and σ_b
  - Correlation of returuns: $\rho_{A,B}$
  - Weights in portfolio: w and $1 - w$
- We can compute expected return μ_w and volatility σ_w as:
  - $\mu_w = w\mu_A + (1 - w)\mu_B$
  - $\sigma_w = \sqrt{w^2\sigma_A^2 + 2w(1 - w)\rho_{A,B}\sigma_A\sigma_B(1 - w)^2\sigma_A^2}$
- For an ***n*-asset portfolio** with vector of weights $\boldsymbol{w}$ it would be:
  - $\mu_w = \boldsymbol{w}'\boldsymbol{\mu}$
  - $\sigma_w = \sqrt{\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}$
- Where $\boldsymbol{\mu}$ is vector of expected returns and $\boldsymbol{\Sigma}$ covariance matrix

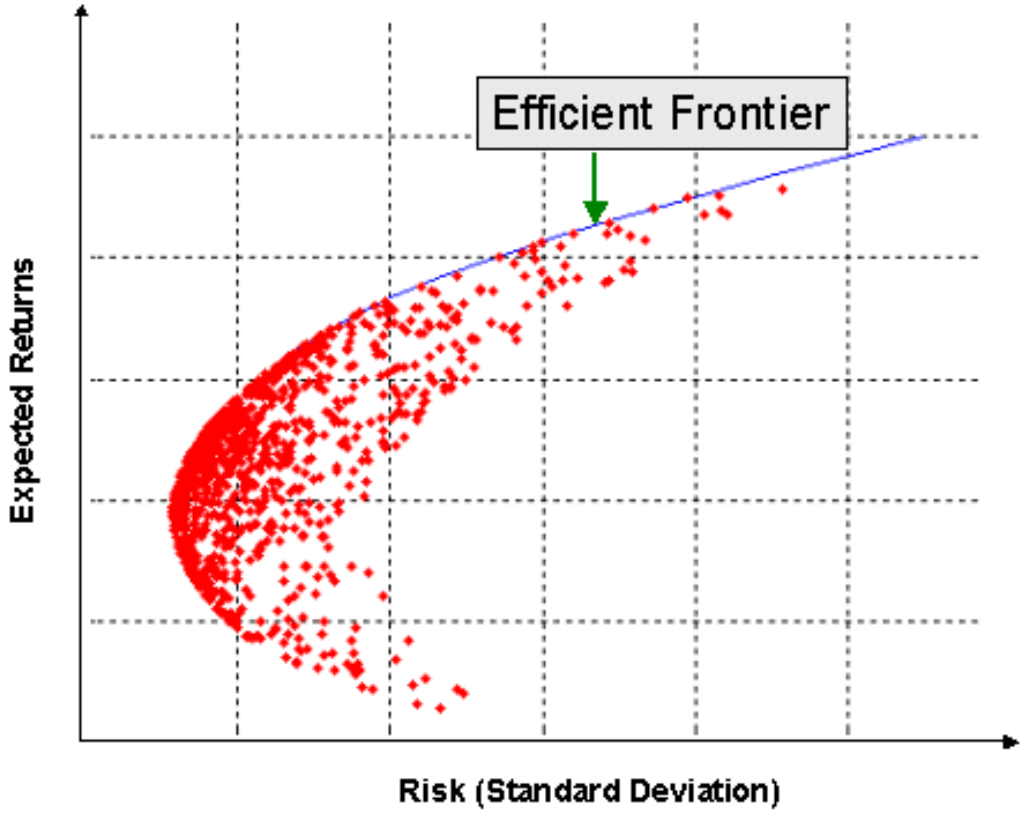
Two-asset portfolio

- Diversification allows us to reduce portfolio risk σ, while keeping the expected return μ the same
- The diversification effect is stronger if ρ is lower



n-asset portfolio

- Diversification effect is stronger if we have more assets
- **Efficient frontier** – Set of portfolios with maximum expected return μ_w for a given level of risk σ_w



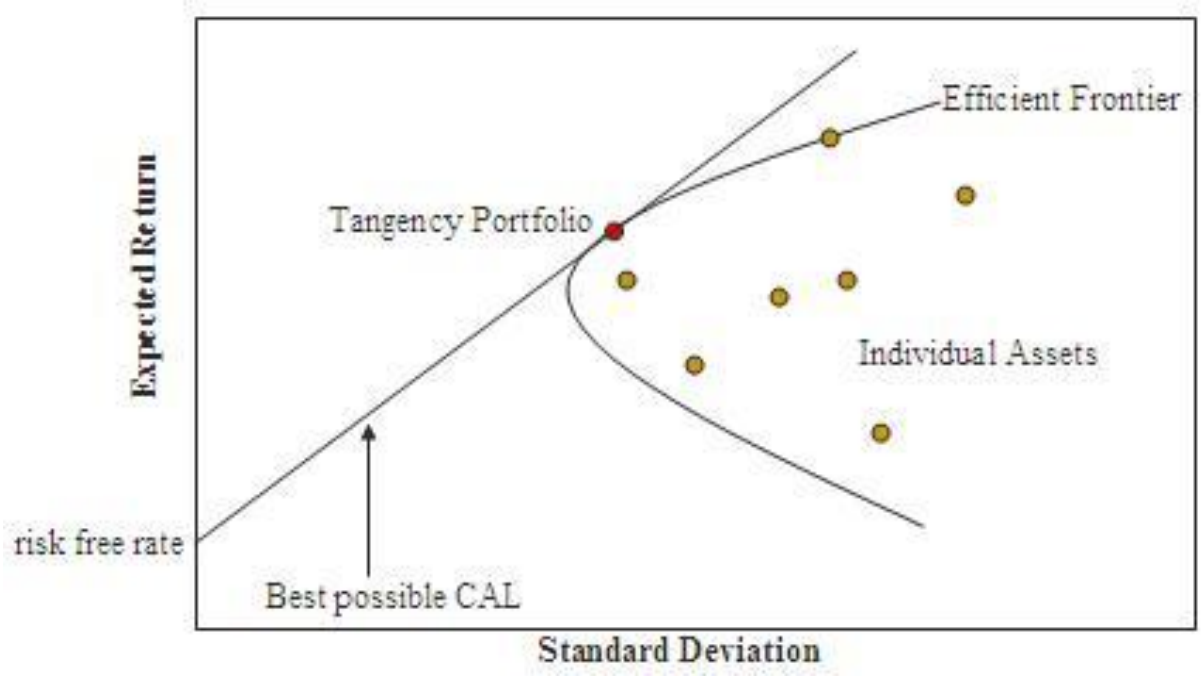
Efficient frontier:

- Can be found by minimizing $\boldsymbol{w'\Sigma w}$ with the constraint $\boldsymbol{w'\mu} = x$
- Can be solved with Lagrange multipliers, leading to a system of linear equations:

$$\begin{bmatrix} 2\boldsymbol{\Sigma} & -\boldsymbol{\mu} \\ \boldsymbol{\mu'} & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{w} \\ \lambda \end{bmatrix} = \begin{bmatrix} 0 \\ x \end{bmatrix}$$

Including a risk-free asset

- All rational investors will hold a combination of the risk-free asset and the **market portfolio** (tangency portfolio)
- Results in maximization of the Sharpe ratio: $Sharpe = \frac{\mathrm{E}(R_P) - R_f}{\sigma_P}$



- The efficient frontier is given by the **Capital Alocation Line:**

$$\mathrm{E}(R_P) = R_f + \sigma_P \frac{\mathrm{E}(R_M) - R_f}{\sigma_M}$$

CAPM model

- Treynor (1961), Sharpe (1964), Lintner (1965), Mossin (1966)
- Solves the problem of asset pricing in the MPT framework
- Equilibrium asset returns depend on their contribution to the total risk of the market portfolio
- Historical asset returns can be decomposed as:

$$R_{i,t} - R_f = \alpha_i + \beta_i \left[R_{M,t} - R_f \right] + \epsilon_{i,t}$$

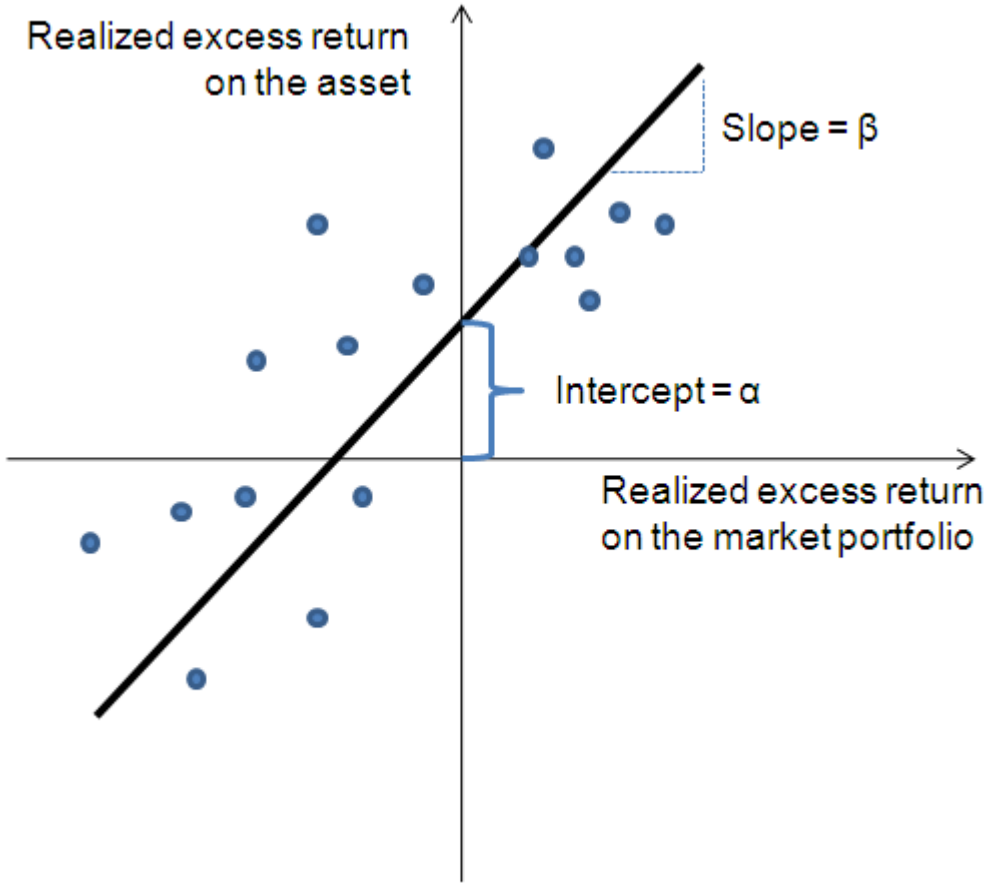
- Where $R_{M,t}$ is the market portfolio return and $\epsilon_{i,t} = N(0, \sigma_i)$
- The total variability of $R_{i,t} - R_f$ is thus decomposed into:
  - **Systematic risk**, equal to β_i, i.e. the sensitivity to $\left[R_{M,t} - R_f \right]$ (risk factor)
  - **Specific risk**, equal to σ_i (which can be diversified away)
- Investors recieve risk premium only for **systematic risk**
- **Expected returns** of any asset are thus given by the equation:

$$\mathrm{E}(R_i) = R_f + \beta_i \left[\mathrm{E}(R_M) - R_f \right]$$

Security Characteristic Line (SCL)

- Historical asset returns are decomposed with:

$$R_{i,t} - R_f = \alpha_i + \beta_i\left[R_{M,t} - R_f\right] + \epsilon_{i,t}$$

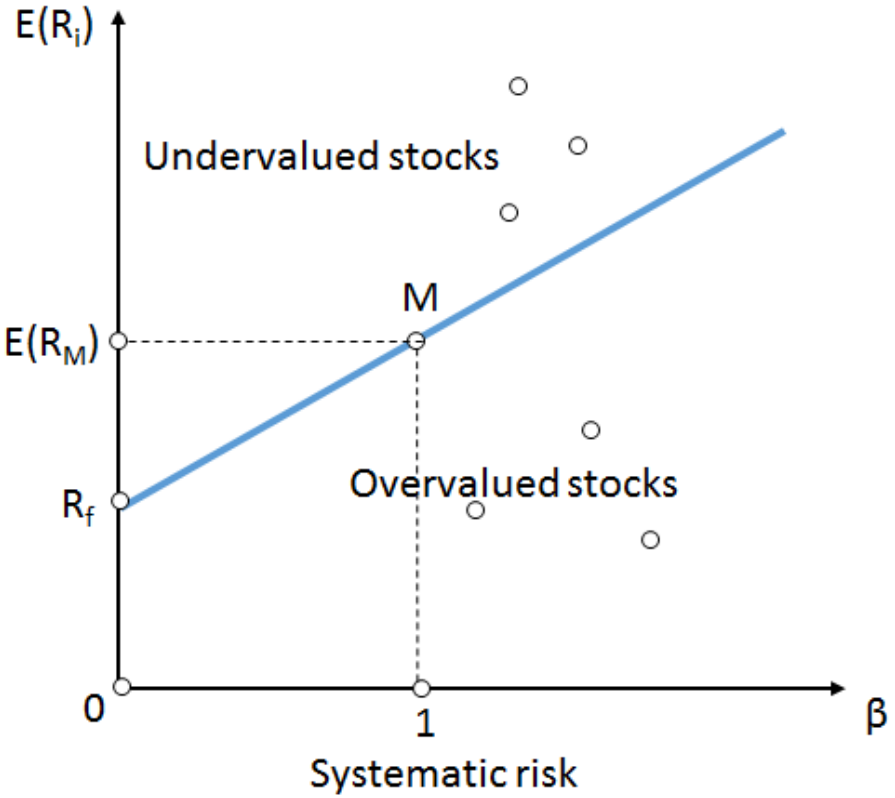


$$\beta_i = \frac{\mathrm{Cov}(R_i, R_M)}{\mathrm{Var}(R_M)}$$

Security Market Line (SML)

- Expected returns of any asset in the CAPM model are:

$$\mathrm{E}(R_i) = R_f + \beta_i\left[\mathrm{E}(R_M) - R_f\right]$$



CAPM valuation:

- $V_0 = \sum_{t=1}^{T} e^{-r_d t} E(CF_t)$
- $r_d = \mathrm{E}(R_i)$
- $\mathrm{E}(R_i)$ = Expected equilibrium return
- CF_t = Expected future dividends

Problems of MPT and CAPM

- Implementation of MPT for portfolio optimization:
  - How to determine the expected asset returns $\boldsymbol{\mu}$?
  - How to exted the model with our own expectations?
  - How to capture the time-variability of $\boldsymbol{\Sigma}$?
- Implementation of CAPM for asset pricing:
  - How to define the market portfolio?
  - How to get the expected return of the market portfolio?
  - How to capture the time-variability of β_i?
  - What about the other risk factors / risk premiums?
- Unrealistic assumptions of MPT and CAPM:
  - Asset returns do not follow the Gaussian distribution
  - Tail-correlation of asset returns
  - Risk-aversion is asymmetric in practice (investors care mostly about the downside risk)

Portfolio optimization and CAPM

- Modern Portfolio Theory (MPT)
- **Portfolio optimization in practice**
- Case study 1 – MPT application on ETFs
- Case study 2A – MPT application on stocks
- Case study 2B – MPT application on stocks with CAPM

Portfolio optimization

- **Portfolio construction** → Combining multiple assets into a portfolio in order to achieve certain objectives regarding the expected returns, risk, tail-risk, factor sensitivities, etc. → can be based on **optimization methods** or **heuristic methods**
- Can be **aplied to any asset class** including **individual strategies:**
  - Portfolios of stocks, bonds, commodities, ETFs, multi-asset, etc.
  - Portfolios of quintile-portfolios, technical trading strategies, etc.
- Goal si to find a vector of asset weights $\boldsymbol{w}$, based on inputs such as the vector of expected returns $\boldsymbol{\mu}$, covariance matrix of asset returns $\boldsymbol{\Sigma}$, factor sensitivities $\boldsymbol{B}$, etc.
- The estimation of the inputs ($\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$, $\boldsymbol{B}$) is the **most problematic part → Markowitz approach** uses **fixed (sample) estimates** of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, which:
  - **Ignores the variability** of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ → Can be solved by estimating them as dynamic $\boldsymbol{\mu}_t$ (VARIMA, VECM, HMM atd.) and $\boldsymbol{\Sigma}_t$ (DCC-GARCH, GARCH-BEKK, etc.)
  - **Ignores the uncertainty of the estimates** → Can be solved with shrinkage methods, LASSO, robust estimation methods, Bayesian estimates, etc.
  - **Historical estimates may not be appropriate for $\boldsymbol{\mu}$** → For ETFs, quantile portfolios, etc. the historical mean is mostly adequate (for long horizons) → For individual stocks, commodities, etc. it is problematic → Alternative is to estimate $\boldsymbol{\mu}$ with some factor model (Fama-French, etc.) or with econometric methods (VARIMA, VECM, HMM, FFNN, LSTM, etc.)

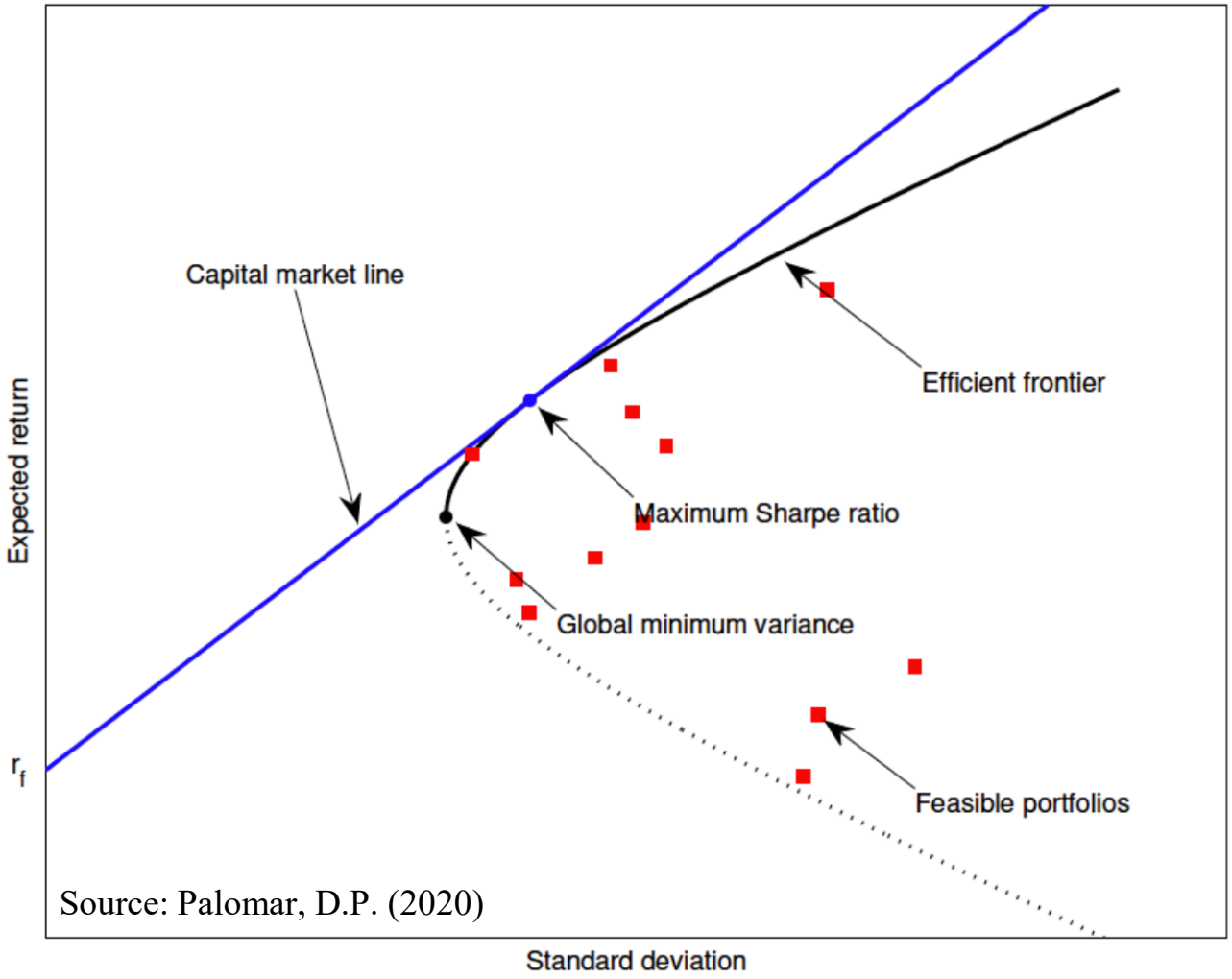
Heuristic portfolios

- **Diversification** → By combining multiple assets into a portfolio we can achieve higher expected return without higher risk (or decrease the risk without decreasing the return)
- **Portfolio optimization** → Finding weights $\boldsymbol{w}$ to get a portfolio with optimal risk/return characteristics (e.g. maximize the return for a given level of risk)
  → Difficult in practice due to **uncertainty about the estimates** of $\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$, etc.
- **Heuristic portfolios** → Simple alternatives to portfolio optimization that omitt the information from $\boldsymbol{\Sigma}$ (and usually $\boldsymbol{\mu}$ too) → Used as benchmark in many studies:
  - **Equally weighted portfolio** → $\boldsymbol{w} = \mathbf{1}/N$ → Optimal in the case of no correlation ($\boldsymbol{\Omega} = \boldsymbol{I}$) and equal volatilities and expected returns ($\boldsymbol{\sigma} = \sigma\mathbf{1}$ and $\boldsymbol{\mu} = \mu\mathbf{1}$) for all assets
    → Often difficult to beat in practice with more advanced methods
  - **Value weighted portfolio** → $\boldsymbol{w} \propto 1/\boldsymbol{V}$, where $\boldsymbol{V}$ denotes the market capitalization vector
    → Would be equivalent to the MPT tangency portfolio if all investors were rational and homogenous (i.e. if everyone held the tangency portfolio)
  - **Naive risk-parity portfolio** → $\boldsymbol{w} \propto 1/\boldsymbol{\sigma}$, i.e. „inverse-volatility weighted portfolio“
    → Optimal in the case of $\boldsymbol{\Omega} = \boldsymbol{I}$ and $\boldsymbol{\mu} = \mu\mathbf{1}$ but different volatilities $\boldsymbol{\sigma}$
    → Used as benchmark when multiple asset classes are involved (stocks, bonds, commodities)
    → Alternatively $1/\boldsymbol{\sigma}^2$ can also be used (or any other risk measure)
- Weighting based on $\boldsymbol{\mu}$ is also possible, e.g. $\boldsymbol{w} \propto 1/\boldsymbol{\mu}$, or $\boldsymbol{w} \propto \boldsymbol{\mu}/\boldsymbol{\sigma}$, or $\boldsymbol{w} \propto \boldsymbol{\mu}/\boldsymbol{\sigma}^2$
  → Suitable for assets where historical $\boldsymbol{\mu}$ predicts well the future (i.e. index ETFs)

Markowitz model

- **Markowitz model** → Returns are *i.i.d.* Gaussian, i.e.: $\boldsymbol{r_t} = N(\boldsymbol{\mu}, \boldsymbol{\Sigma}), i.i.d.$
- Portfolio return is then $\boldsymbol{r_w} = N(\mu_w, \sigma_w)$ where $\mu_w = \boldsymbol{w'\mu}$ and $\sigma_w = \sqrt{\boldsymbol{w'\Sigma w}}$
  - **Efficient frontier** → Pareto efficient frontier of **risky-asset portfolios** that:
    1. Maximize μ_w for a given σ_w: $\max_{\boldsymbol{w}} \boldsymbol{w'\mu} \mid \boldsymbol{w'\Sigma w} = c, \boldsymbol{w'1} = 1$
    2. Minimize σ_w for a given μ_w: $\min_{\boldsymbol{w}} \boldsymbol{w'\Sigma w} \mid \boldsymbol{w'\mu} = c, \boldsymbol{w'1} = 1$
    3. Maximize $\mu_w - \lambda\sigma_w$: $\max_{\boldsymbol{w}} \boldsymbol{w'\mu} - \lambda \boldsymbol{w'\Sigma w} \mid \lambda = c, \boldsymbol{w'1} = 1$
  - Where λ is **degree risk-aversion** and $\boldsymbol{w'1} = 1$ is the the **budget constraint**
  - The three formulations are **equivalent** and under no additional constrains (apart from the budget constrain $\boldsymbol{w'1} = 1$) allow for analytical solution
  - **Solution for 3. →** $\boldsymbol{w} = \frac{1}{2\lambda}\boldsymbol{\Sigma^{-1}}(\boldsymbol{\mu} + v\boldsymbol{1}), \quad v = \frac{2\lambda - \boldsymbol{1'\Sigma^{-1}\mu}}{\boldsymbol{1'\Sigma^{-1}1}}$ (for any λ)
  - **Minimum variance portfolio** → Portfolio of **risky assets** with minimum σ_w
    - $\min_{\boldsymbol{w}} \boldsymbol{w'\Sigma w} \mid \boldsymbol{w'1} = 1$ → Solution → $\boldsymbol{w} = \frac{\boldsymbol{\Sigma^{-1}1}}{\boldsymbol{1'\Sigma^{-1}1}}$
  - **Tangency portfolio** → Portfolio that maximizes the **Sharpe ratio** → Defines the **efficient frontier** (Capital Market Line) if a **risk-free asset is allowed**
    - $\max_{\boldsymbol{w}} \frac{\boldsymbol{w'\mu} - r_f}{\sqrt{\boldsymbol{w'\Sigma w}}} \mid \boldsymbol{w'1} = 1$ → Solution → $\boldsymbol{w} = \frac{\boldsymbol{\Sigma^{-1}}(\boldsymbol{\mu} - r_f\boldsymbol{1})}{\boldsymbol{1'\Sigma^{-1}}(\boldsymbol{\mu} - r_f\boldsymbol{1})}$

Efficient frontier – Illustration



- **Dual fund theorem** → Once the risk-free asset is allowed, the Efficient frontier becomes equal to a line (Capital Market Line) → All investors should hold a combination of the risk-free asset and the Tangency portfolio (upper part of the CML can be attained by borrowing funds fo the risk-free rate and investing into the Tangency portfolio with leverage)

Minimum variance portfolio

Minimum variance portfolio – Proof of the solution

- We need to solve the minimization problem: $\min_{\boldsymbol{w}} \boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w} \mid \boldsymbol{w}'\mathbf{1} = 1$
- Where $\sigma_{\boldsymbol{w}}^2 = \boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}$ is portfolio variance and $\boldsymbol{w}'\mathbf{1} = 1$ the budget constraint
- The **Lagrangian** of the problem is: $L(\boldsymbol{w}, \lambda) = \boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w} + \lambda(\boldsymbol{w}'\mathbf{1} - 1)$
- The first order conditions (FOCs) for a minimum are:

$$\frac{\partial L(\boldsymbol{w},\lambda)}{\partial \boldsymbol{w}} = \frac{\partial \boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}{\partial \boldsymbol{w}} + \frac{\partial \lambda(\boldsymbol{w}'\mathbf{1}-1)}{\partial \boldsymbol{w}} = 2\boldsymbol{\Sigma}\boldsymbol{w} + \lambda\mathbf{1} = \mathbf{0}$$

$$\frac{\partial L(\boldsymbol{w},\lambda)}{\partial \lambda} = \frac{\partial \boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}{\partial \lambda} + \frac{\partial \lambda(\boldsymbol{w}'\mathbf{1}-1)}{\partial \lambda} = \boldsymbol{w}'\mathbf{1} - 1 = 0$$

- We express $\boldsymbol{w}$ from the 1st FOC: $\boldsymbol{w} = -\frac{1}{2}\lambda\boldsymbol{\Sigma}^{-1}\mathbf{1}$
- Then multiply both sides with $\mathbf{1}'$: $\mathbf{1}'\boldsymbol{w} = -\frac{1}{2}\lambda\mathbf{1}'\boldsymbol{\Sigma}^{-1}\mathbf{1}$
- According to the 2nd FOC: $\mathbf{1}'\boldsymbol{w} = 1$
- Allowing us to solve for λ: $\lambda = -2\frac{1}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}\mathbf{1}}$
- Substitute λ back into the 1st FOC: $\boldsymbol{w} = -\frac{1}{2}\left(-2\frac{1}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}\mathbf{1}}\right)\boldsymbol{\Sigma}^{-1}\mathbf{1}$
- Which gives us the **solution**: $\boldsymbol{w} = \frac{\boldsymbol{\Sigma}^{-1}\mathbf{1}}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}\mathbf{1}}$

Minimum variance portfolio

❖ **Minimum variance portfolio – Proof of the solution (illustration)**

- We need to solve the maximization problem: $\max_{\boldsymbol{w}} \frac{\boldsymbol{w}'\boldsymbol{\mu}-r_f}{\sqrt{\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}} \mid \boldsymbol{w}'\mathbf{1} = 1$
- Where $\frac{\boldsymbol{w}'\boldsymbol{\mu}-r_f}{\sqrt{\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}}$ is portfolio **Sharpe ratio** and $\boldsymbol{w}'\mathbf{1} = 1$ the **budget constraint**
- The **Lagrangian** is: $L(\boldsymbol{w}, \lambda) = \left(\boldsymbol{w}'\boldsymbol{\mu} - r_f\right)(\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w})^{-\frac{1}{2}} + \lambda(\boldsymbol{w}'\mathbf{1} - 1)$
- The **first order conditions (FOCs)** for a maximum are:

$$\frac{\partial L(\boldsymbol{w},\lambda)}{\partial \boldsymbol{w}} = \boldsymbol{\mu}(\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w})^{-\frac{1}{2}} - \left(\boldsymbol{w}'\boldsymbol{\mu} - r_f\right)(\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w})^{-\frac{3}{2}}\boldsymbol{\Sigma}\boldsymbol{w} + \lambda\mathbf{1} = \mathbf{0}$$

$$\frac{\partial L(\boldsymbol{w},\lambda)}{\partial \lambda} = \boldsymbol{w}'\mathbf{1} - 1 = 0$$

- Which can be solved to get the **solution**: $\boldsymbol{w} = \frac{\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}-r_f\mathbf{1})}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}-r_f\mathbf{1})}$
- For proofs for some other variants/constrains see Zivot, E. (2021)
- **Note:** Without the budget constraint you may get: $\boldsymbol{w} = \frac{1}{\gamma}\boldsymbol{\Sigma}^{-1}\left(\boldsymbol{\mu} - r_f\mathbf{1}\right)$
  - Which is equal to the budget constrained solution for $\gamma = \mathbf{1}'\boldsymbol{\Sigma}^{-1}\left(\boldsymbol{\mu} - r_f\mathbf{1}\right)$, while for other values of parameter γ it gives other portfolios on the CML line

Portfolio optimization and CAPM

- Modern Portfolio Theory (MPT)
- Portfolio optimization in practice
- **Case study 1 – MPT application on ETFs**
- Case study 2A – MPT application on stocks
- Case study 2B – MPT application on stocks with CAPM

Study 1: ETF portfolio optimization

- **Task:** Download the data for the ETFs (i) SPY, (ii) VNQ and (iii) TLT, find the tangency portfolio, minimum variance portfolio, the efficient frontier and the capital market line + plot the realized returns of the optimized portfolios on a chart
- **SPY** = SPDR S&P 500 ETF Trust, **VNQ** = Vanguard Real Estate Index Fund, and **TLT** = iShares 20+ Year Treasury Bond ETF
- **Steps:**
  1. Download OHLCV data for the SPY, VNQ and TLT
  2. Match the beginning of the ETF time series
  3. Convert the dataset to monthly frequency and compute the returns
  4. Convert monthly returns into a matrix
  5. Compute vector of mean returns $\boldsymbol{\mu}$ and covariance matrix of returns $\boldsymbol{\Sigma}$
  6. Find the tangency portfolio and the minimum variance portfolio
  7. Construct the efficient frontier (composed of risky assets)
  8. Construct the capital market line
  9. Plot realized returns of the optimized portfolios on a chart

ETF portfolio optimization (1)

1. Download OHLCV data for the SPY, VNQ and TLT

```
# Load tidyquant
library(tidyverse)
library(tidyquant)
# Settings:
symbols = c("SPY","VNQ","TLT")
tStart <- "1990-01-01"
tEnd   <- today()
# Load ETF prices
data <- tq_get(symbols, get = "stock.prices", from = tStart, to = tEnd)
```

2. Match the beginning of the ETF time series

```
# Find the common dataset start
startDate <- data %>%
  group_by(symbol) %>%
  summarize(date = min(date))
# Filter the dataset
tStart <- max(startDate$date)
data2 <- data %>%
  filter(date>=tStart)
```

ETF start dates:

```
# A tibble: 3 × 2
  symbol date
  <chr>  <date>
1 SPY    1993-01-29
2 TLT    2002-07-30
3 VNQ    2004-09-29
```

Note: The start dates of the ETFs in tibble **data** differ, depending on the ETF (see above), as SPY was introduced in 1993, TLT in 2002 and VNQ in 2004

The modified tibble **data2** removes all of the data before 2004-09-29, making the lenght of the price time series the same for all three ETFs

ETF portfolio optimization (2)

3. Convert the dataset to monthly frequency and compute the returns

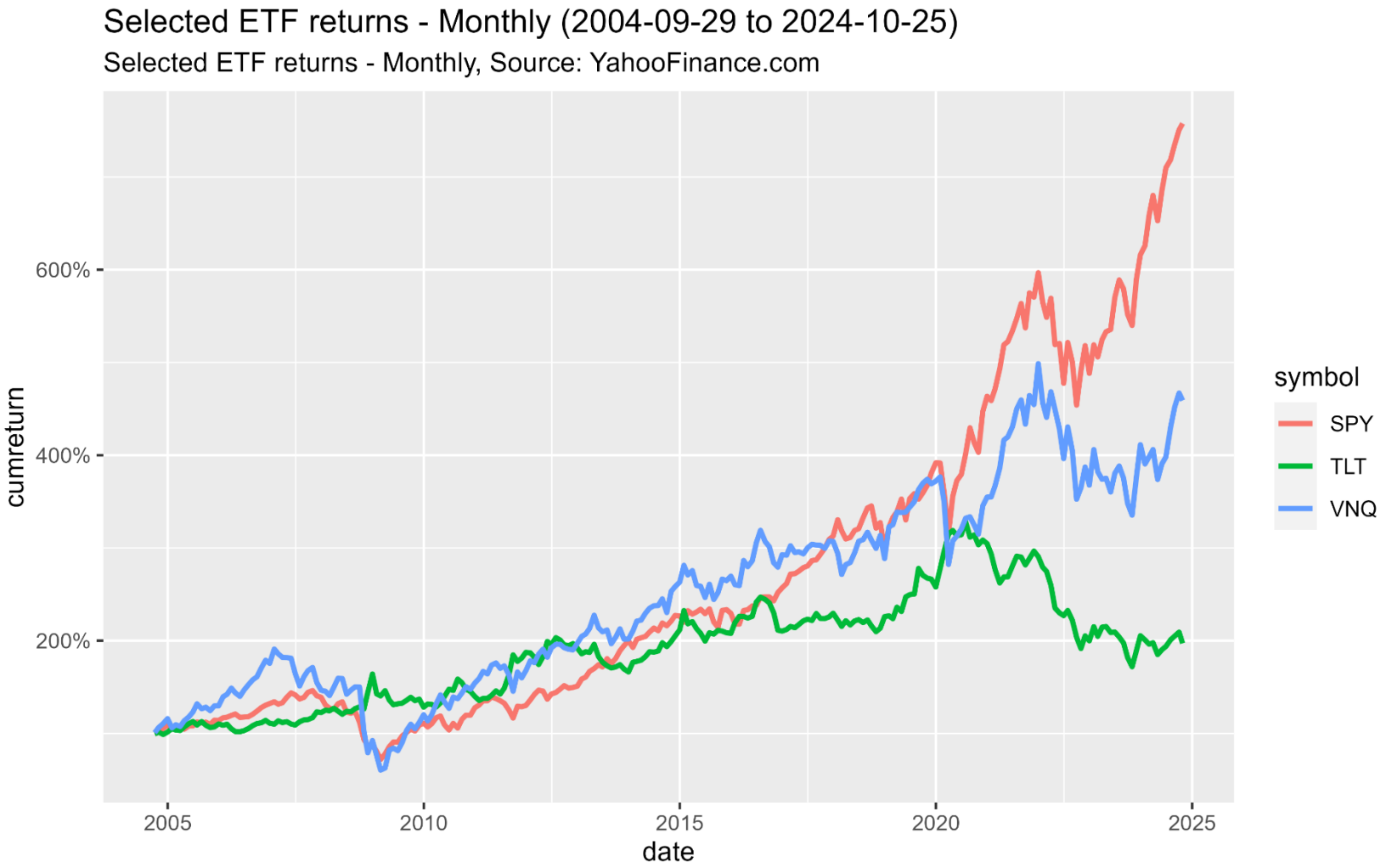
```
# Compute daily log-returns
data2 <- data2 %>%
  group_by(symbol) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice,1),
         return = ifelse(is.na(return),0,return))
# Convert to monthly
mdata <- data2 %>%
  mutate(year = year(date),
         month = month(date)) %>%
  group_by(symbol,year,month) %>%
  summarize(date = max(date),
            return = sum(return)) %>%
  ungroup() %>%
  mutate(returnS = exp(return)-1)
# Plot cumulative monthly returns on chart
gChart03 <- mdata %>%
  group_by(symbol) %>%
  mutate(cumreturn = cumprod(1+returnS)) %>%
  ggplot(aes(x = date, y = cumreturn, color = symbol)) +
  geom_line(size = 1) +
  labs(title = paste0("Selected ETF returns - Monthly (",dStart," to ",dEnd,")"),
       subtitle = paste0("ETF returns - Monthly, Source: YahooFinance.com")) +
  scale_y_continuous(labels = scales::percent)
print(gChart03)
```

Note: We first compute the daily log-returns as they can easily be aggregated to a monthly frequency with simple summation.
The monhtly log-returns can then be converted to simple returns with **exp(return)-1**

Chart: Before plotting the chart, we should also run:

```
# Dataset bounds
dStart <- min(data2$date)
dEnd <- max(data2$date)
```

Cumulative ETF returns (N=3)



SPY = SPDR S&P 500 ETF Trust → Tracks US stock market

TLT = iShares 20+ Year Treasury Bond ETF → Tracks US government bond market

VNQ = Vanguard Real Estate Index Fund → Tracks US housing market

ETF portfolio optimization (3)

4. Convert monthly returns into a matrix

```
# Extract return data
mreturns <- mdata %>%
  select(symbol,date,returnS) %>%
  spread(symbol,returnS) %>%
  select(date,symbols)
# Convert to matrix
ret <- mreturns %>%
  select(symbols)
ret <- as.matrix(ret)
```

Note: In our example we pefrom the portfolio optimization based on simple returns (**returnS**), an alternative would be to use log-returns, which are closer to Gaussian, but the aggregation to portfolio-level would then be slightly inaccurate

5. Compute vector of mean returns μ and covariance matrix of returns Σ

```
# Compute parameters
mju <- colMeans(ret)
covmat <- cov(ret)
cormat <- cor(ret)
sigma <- diag(covmat)^(1/2)
```

Commentary: SPY provides average return 0.93% p.m. with volatility 4.3% p.m., VNQ has higher avg. return with higher volatility, while TLT has lower avg. return with lower volatility. SPY is strongly positively correlated with VNQ but negatively correlated with TLT

ETF return statistics:

```
> round(mju,4)
   SPY    VNQ    TLT
0.0093 0.0084 0.0036

> round(sigma,4)
   SPY    VNQ    TLT
0.0430 0.0635 0.0398

> round(cormat,4)
        SPY    VNQ    TLT
SPY  1.0000 0.7452 -0.105
VNQ  0.7452 1.0000  0.120
TLT -0.1050 0.1200  1.000
```

ETF portfolio optimization (4)

6. Find the tangency portfolio and the minimum variance portfolio

```
# Compute benchmark portfolios:
w_ew <- rep(1/length(mju),length(mju)) # Equal-weight portfolio
w_rp <- (1/sigma)/sum(1/sigma) # Naive risk-parity portfolio
# Compute Markowitz portfolios:
rf <- 0.03/12 # We assume risk-free interest rate 3% p.a.
w_mv <- (ginv(covmat) %*% rep(1,length(mju)))/sum(ginv(covmat) %*% rep(1,length(mju)))
w_ta <- (ginv(covmat) %*% (as.vector(mju)-rf))/sum(ginv(covmat) %*% (as.vector(mju)-rf))
```

Note: We use **ginv** function from package MASS instead of R native function **solve** to compute the matrix inverse

The calculated portfolios are:

- **Equal Weighted portfolio:** Heuristic portfolio that allocates $\boldsymbol{w} = \mathbf{1}/N$
- **Naive Risk Parity:** Heuristic portfolio that allocates $\boldsymbol{w} \propto 1/\boldsymbol{\sigma}$
- **Minimum variance:** Optimized minimum variance portfolio → $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}\mathbf{1}$
- **Tangency portfolio:** Optimized tangency portfolio (max. Sharpe) → $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})$

Note: The minVar and Tangency portfolios found in this way allow for negative weights (i.e. short positions), with the only constrain being $\boldsymbol{w}'\mathbf{1} = 1$

Illustration → Save optimized portfolio weights into a tibble:

```
# Compute portfolio weight matrix
port_weights <- as.tibble(rbind(w_ew,w_rp,t(w_mv),t(w_ta)))
port_weights$strategy <- c_port
port_weights <- port_weights %>%
  relocate(strategy)
port_weights
```

ETF portfolio optimization (5)

Illustration 2 → Compute risk-return characteristics of the optimized portfolios:

```
# Compute portfolio risk-return characteristics
m_ew <- mju %*% w_ew
m_rp <- mju %*% w_rp
m_mv <- mju %*% w_mv
m_ta <- mju %*% w_ta
v_ew <- sqrt(w_ew %*% covmat %*% w_ew)
v_rp <- sqrt(w_rp %*% covmat %*% w_rp)
v_mv <- sqrt(as.vector(w_mv) %*% covmat %*% as.vector(w_mv))
v_ta <- sqrt(as.vector(w_ta) %*% covmat %*% as.vector(w_ta))
c_port <- c("Equal_Weighted","Naive_Risk_Parity","Minimum_Variance","Tangency_Portfolio")
mju_vec <- c(m_ew,m_rp,m_mv,m_ta)
sigma_vec <- c(v_ew,v_rp,v_mv,v_ta)
port_char <- tibble(strategy = c_port, mju = mju_vec, sigma = sigma_vec)
port_char$rf <- rf
port_char$sharpe <- (port_char$mju - port_char$rf)/port_char$sigma
port_char$sharpe_pa <- port_char$sharpe * sqrt(12)
# Add ETF risk-return characteristics
stock_char <- tibble(strategy = symbols, mju = mju, sigma = sigma)
stock_char$rf <- rf
stock_char$sharpe <- (stock_char$mju - stock_char$rf)/stock_char$sigma
stock_char$sharpe_pa <- stock_char$sharpe * sqrt(12)
# Combined ETF and portfolio risk-return characteristics + risk-free asset
port_char2 <- rbind(stock_char,port_char)
temp <- tibble(strategy="Risk_Free",mju=rf, sigma=0, rf=rf, sharpe=NA, sharpe_pa=NA)
port_char3 <- rbind(port_char2,temp)
```

Optimal portfolios (ETF)

Weights of individual ETFs in the optimal portfolios:

```
> port_weights
# A tibble: 4 × 4
  strategy              SPY    VNQ   TLT
  <chr>               <dbl>  <dbl> <dbl>
1 Equal_Weighted      0.333  0.333 0.333
2 Naive_Risk_Parity   0.363  0.246 0.392
3 Minimum_Variance    0.643 -0.193 0.550
4 Tangency_Portfolio  0.963 -0.242 0.280
```

Note: See that the minVar and the Tangency portfolios recommend shorting VNQ so that we can invest into SPY and TLT with leverage

Risk-return characteristics of the portfolios and ETFs:

```
> port_char3
# A tibble: 8 × 6
  strategy              mju  sigma     rf  sharpe sharpe_pa
  <chr>               <dbl>  <dbl>  <dbl>   <dbl>     <dbl>
1 SPY               0.00933 0.0430 0.0025  0.159     0.551
2 VNQ               0.00839 0.0635 0.0025  0.0928    0.322
3 TLT               0.00359 0.0398 0.0025  0.0274    0.0948
4 Equal_Weighted    0.00711 0.0362 0.0025  0.127     0.441
5 Naive_Risk_Parity 0.00685 0.0332 0.0025  0.131     0.455
6 Minimum_Variance  0.00636 0.0264 0.0025  0.146     0.506
7 Tangency_Portfolio 0.00795 0.0314 0.0025 0.174     0.602
8 Risk_Free         0.0025  0      0.0025  NA        NA
```

Note: Tangency portfolio has the highest possible Sharpe ratio, while the minimum variance portfolio has the lowest possible sigma

ETF portfolio optimization (6)

7. Construct the efficient frontier (composed of risky assets)

```
# Construct efficient frontier
lambda_vec <- 1*(1.1^(0:200))
w_eff <- NULL
mju_eff <- NULL
sigma_eff <- NULL
V1 <- rep(1,length(mju))
for (lambda in lambda_vec) {
  v <- (2*lambda-(V1 %*% ginv(covmat) %*%mju))/(V1 %*% ginv(covmat) %*% V1)
  w <- (1/(2*lambda))*ginv(covmat) %*% (mju + as.numeric(v)*V1)
  w_eff <- rbind(w_eff,as.vector(w))
  mju_eff <- rbind(mju_eff,mju %*% w)
  sigma_eff <- rbind(sigma_eff,sqrt(as.vector(w) %*% covmat %*% as.vector(w)))
}
# Characteristics of the efficient frontier portfolios
port_eff <- tibble(strategy = paste0("lambda_",lambda_vec),
                   mju = as.vector(mju_eff), sigma = as.vector(sigma_eff))
port_eff$rf <- rf
port_eff$sharpe <- (port_eff$mju - port_eff$rf)/port_eff$sigma
port_eff$sharpe_pa <- port_eff$sharpe * sqrt(12)
```

Note: The calculation loops over different values of the mean-reversion parameter λ and applies the formulas:

$$\boldsymbol{w} = \frac{1}{2\lambda}\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} + v\mathbf{1}), \quad v = \frac{2\lambda - \mathbf{1}'\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}}{\mathbf{1}'\boldsymbol{\Sigma}^{-1}\mathbf{1}}$$

ETF portfolio optimization (7)

8. Construct the capital market line

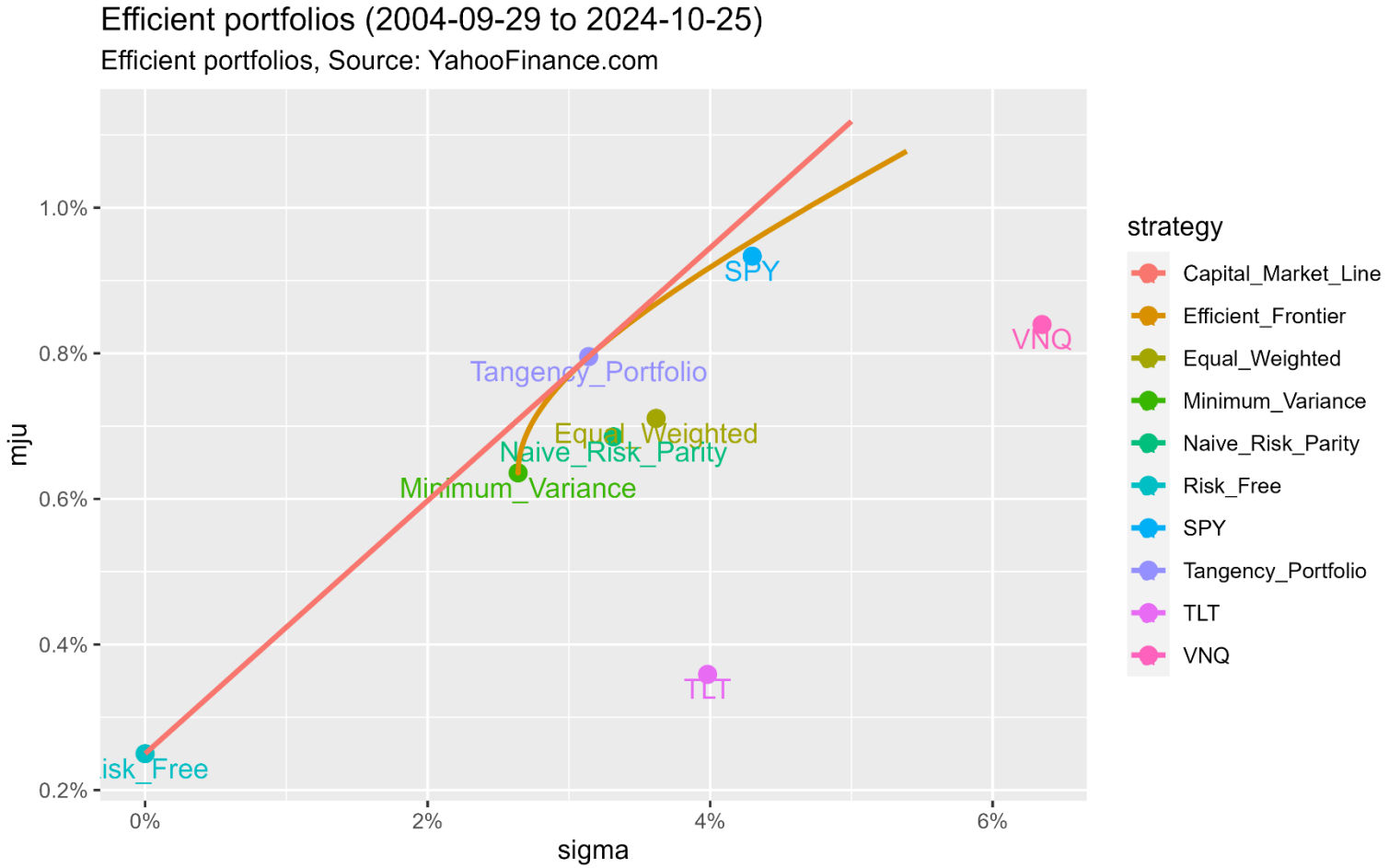
Note: CML intercept is equal to the risk-free rate and its slope is equal to the Sharpe ratio of the Tangency portfolio

```
# Construct capital market line
v_cml_vec <- seq(0,0.05,0.001)
m_cml_vec <- rf + v_cml_vec*((m_ta[1]-rf)/v_ta[1])
# Capital market line portfolios
port_cml <- tibble(strategy="Capital_Market_Line",mju = m_cml_vec,sigma = v_cml_vec)
port_cml$rf <- rf
port_cml$sharpe <- (port_cml$mju - port_cml$rf)/port_cml$sigma
port_cml$sharpe_pa <- port_cml$sharpe * sqrt(12)
```

Illustration → Plot the CML + Efficient Frontier + All portfolios on a chart:

```
# Plot portfolio risk-return characteristics on a chart
temp <- port_eff
temp$strategy = "Efficient_Frontier"
gChart04 <- port_char3 %>%
  ungroup() %>%
  group_by(strategy) %>%
  ggplot(aes(x = sigma, y = mju, color = strategy, label = strategy)) +
  geom_point(size = 3) +
  geom_text(size=4,nudge_y = -0.0002) +
  labs(title = paste0("Efficient portfolios (",dStart," to ",dEnd,")"),
       subtitle = paste0("Efficient portfolios, Source: YahooFinance.com")) +
  scale_x_continuous(labels = scales::percent) +
  scale_y_continuous(labels = scales::percent) +
  geom_line(size = 1, data = temp, aes(x = sigma, y = mju)) +
  geom_line(size = 1, data = port_cml, aes(x = sigma, y = mju))
print(gChart04)
```

Illustration – Efficient frontier (ETF)



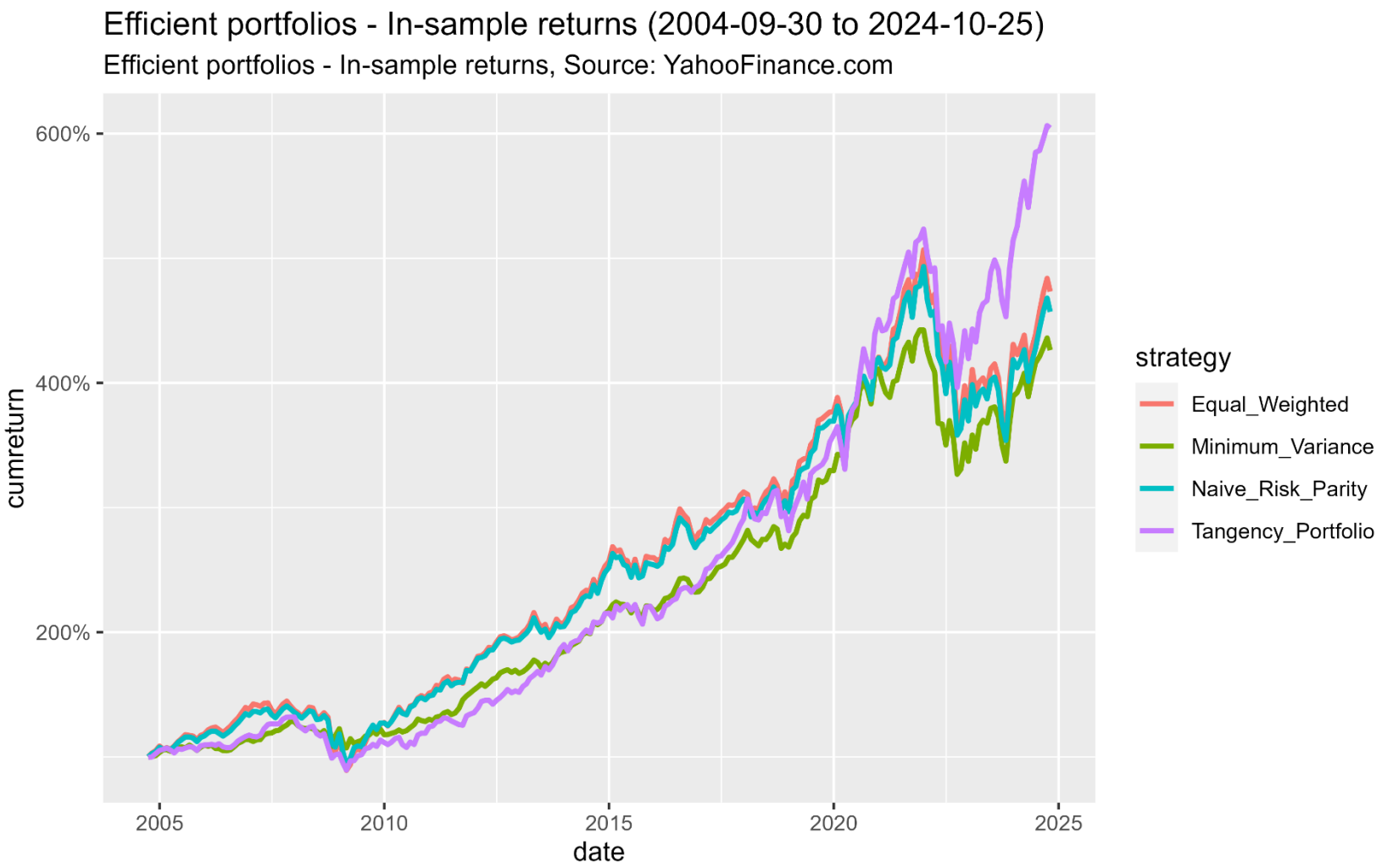
- The compared portfolios are: (portfolios allow for shorts but no leverage: $\boldsymbol{w'1} = 1$)
  - **Equal Weighted portfolio:** Heuristic portfolio that allocates $\boldsymbol{w} = \boldsymbol{1}/N$
  - **Naive Risk Parity:** Heuristic portfolio that allocates $\boldsymbol{w} \propto 1/\boldsymbol{\sigma}$
  - **Minimum variance:** Optimized minimum variance portfolio $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}\boldsymbol{1}$
  - **Tangency portfolio:** Optimized tangency portfolio (max. Sharpe) $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\boldsymbol{1})$

ETF portfolio optimization (7)

9. Plot realized returns of the optimized portfolios on a chart

```
# Compute historical performance
ret_ew <- ret %*% w_ew
ret_rp <- ret %*% w_rp
ret_mv <- ret %*% as.vector(w_mv)
ret_ta <- ret %*% as.vector(w_ta)
# Compute portfolio returns
ret_port <- cbind(ret_ew,ret_rp,ret_mv,ret_ta)
mreturns_port <- as.tibble(ret_port)
names(mreturns_port) <- c_port
mreturns_port <- cbind(mreturns,mreturns_port)
# Convert into long format
mreturns_long <- mreturns_port %>%
  gather(strategy,returnS,-date) %>%
  group_by(strategy) %>%
  mutate(cumreturn = cumprod(1+returnS))
# Plot cumulative returns
gChart06 <- mreturns_long %>%
  filter(strategy %in% c_port) %>%
  ungroup() %>%
  group_by(strategy) %>%
  mutate(cumreturn = cumprod(1+returnS)) %>%
  ggplot(aes(x = date, y = cumreturn, color = strategy)) +
  geom_line(size = 1) +
  labs(title = paste0("Efficient portfolios (",dStart," to ",dEnd,")"),
       subtitle = paste0("Efficient portfolios, Source: YahooFinance.com")) +
  scale_y_continuous(labels = scales::percent)
```

Optimized ETF portfolios (In-Sample)



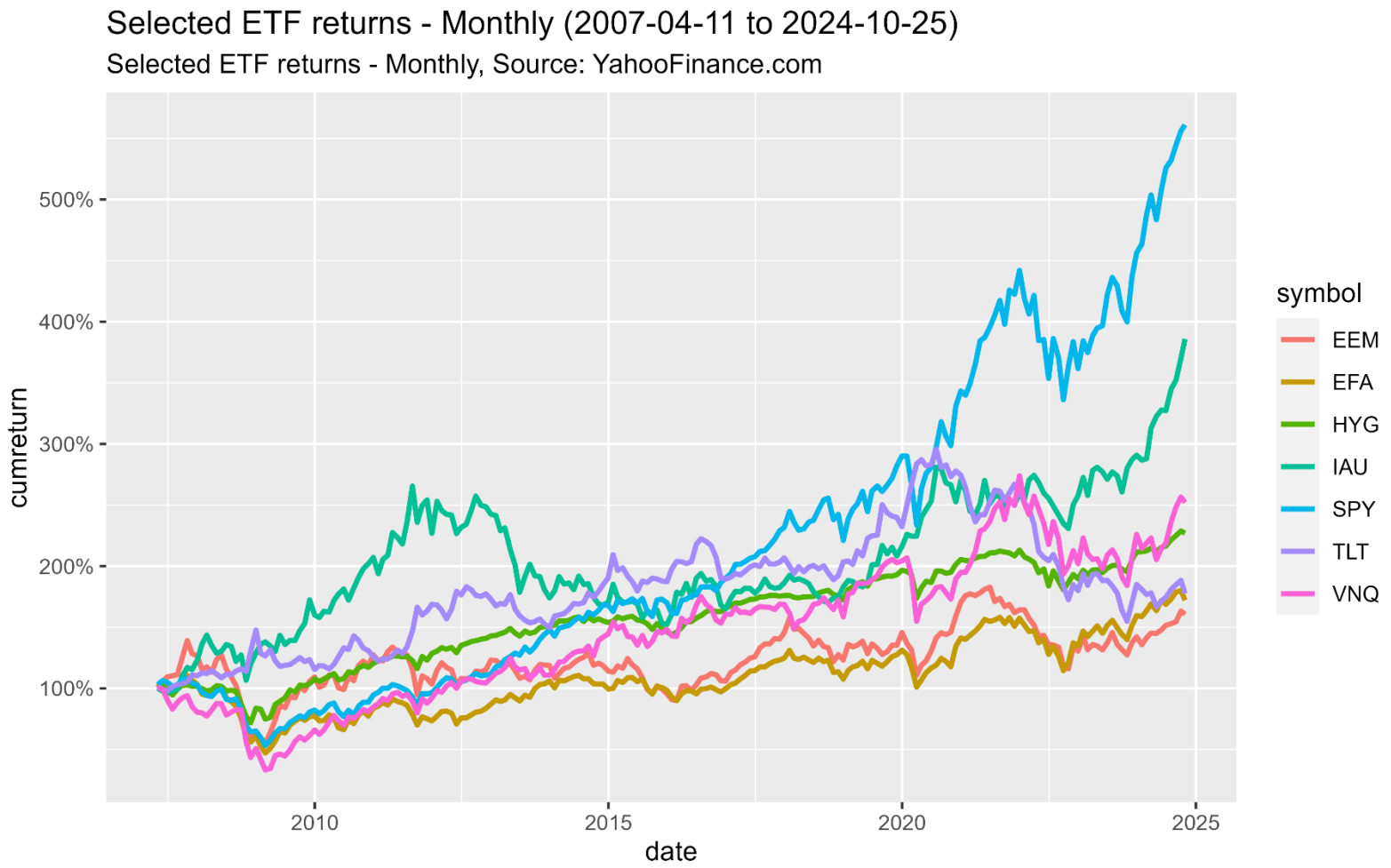
- **Tangency portfolio** exhibits the highest cumulative return
- Differences are modest as we are using only 3 ETFs → Limited benefits from diversification
- Results are also likely to be **over-optimistic** as the back-test is performer **in-sample**

Extension 1.A: More ETFs

- **Task:** Download the data for 7 ETFs (listed below), find the tangency portfolio, minimum variance portfolio, the efficient frontier and the capital market line + plot the realized returns of the optimized portfolios on a chart
- **List of ETFs:**
  - **SPY** = SPDR S&P 500 ETF Trust
  - **EFA** = iShares MSCI EAFE ETF
  - **EEM** = iShares MSCI Emerging Markets ETF
  - **VNQ** = Vanguard Real Estate Index Fund
  - **TLT** = iShares 20+ Year Treasury Bond ETF
  - **IAU** = iShares Gold Trust
  - **HYG** = iShares High Yield Corporate Bond ETF
- **Solution:**
  - We can use the same code as for the original case study
  - Only the data list of symbols changes to:

```
symbols = c("SPY","EFA","EEM","VNQ","TLT","IAU","HYG")
```

Illustration – ETF returns (N=7)



SPY (SPDR S&P 500 ETF Trust), **TLT** (iShares 20+ Year Treasury Bond ETF), **VNQ** (Vanguard Real Estate Index Fund), **EFA** (iShares MSCI EAFE ETF), **EEM** (iShares MSCI Emerging Markets ETF), **IAU** (iShares Gold Trust), **HYG** (iShares High Yield Corporate Bond ETF)

Optimal portfolios (7 ETFs)

Weights of individual ETFs in the optimal portfolios:

```
> port_weights
# A tibble: 4 × 8
  strategy             SPY      EFA     EEM     VNQ   TLT   IAU   HYG
  <chr>              <dbl>    <dbl>   <dbl>   <dbl> <dbl> <dbl> <dbl>
1 Equal_Weighted     0.143   0.143    0.143   0.143 0.143 0.143 0.143
2 Naive_Risk_Parity  0.147   0.130    0.108   0.101 0.160 0.137 0.217
3 Minimum_Variance   0.341  -0.0913  -0.117  -0.257 0.263 0.117 0.745
4 Tangency_Portfolio 1.96   -1.26    -0.127  -0.191 0.104 0.390 0.127
```

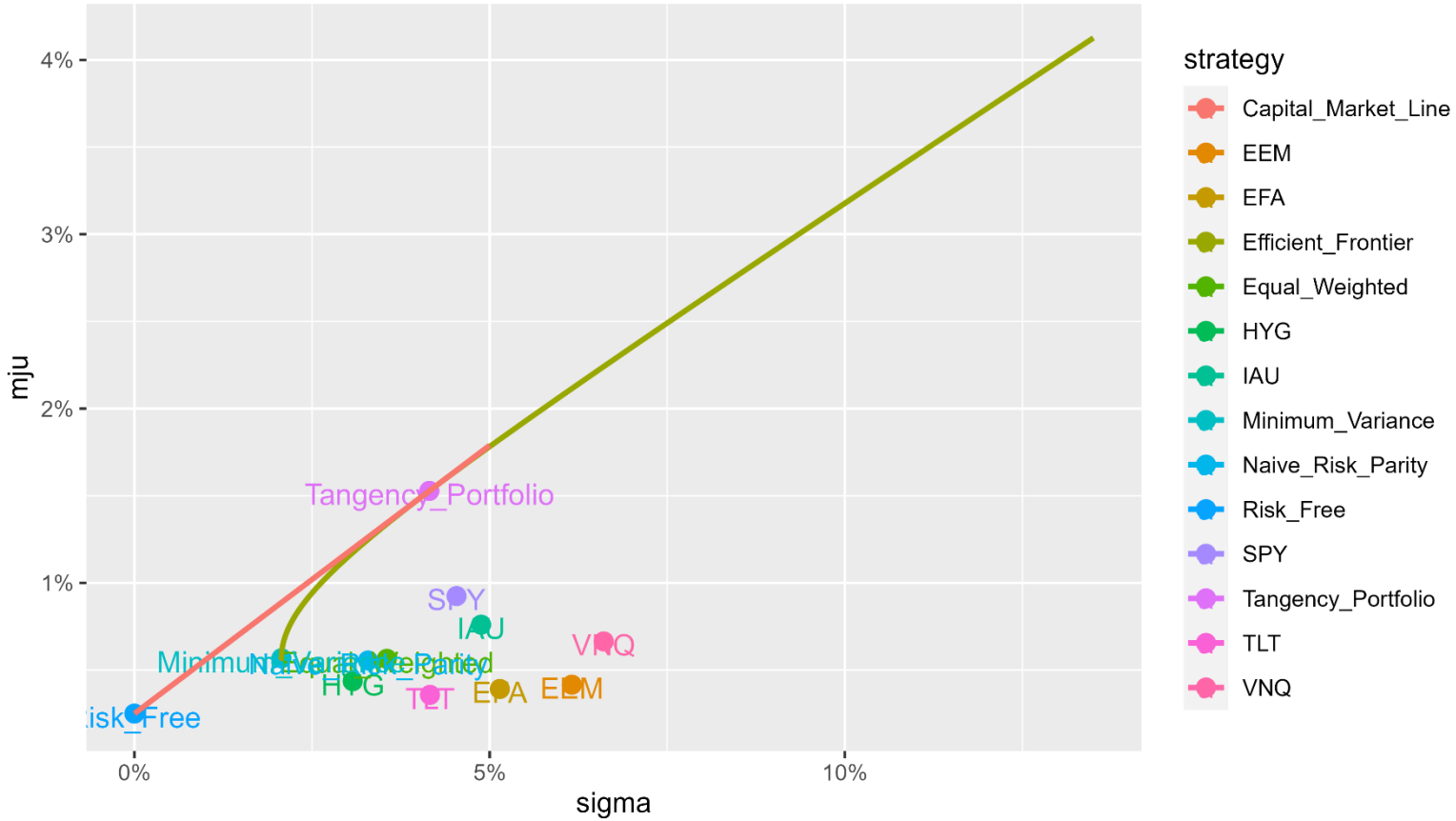
Risk-return characteristics of the portfolios and ETFs:

```
> port_char3
# A tibble: 12 × 6
   strategy              mju   sigma      rf  sharpe sharpe_pa
   <chr>               <dbl>   <dbl>   <dbl>   <dbl>     <dbl>
 1 SPY               0.00924 0.0453  0.0025  0.149      0.515
 2 EFA               0.00391 0.0514  0.0025  0.0275     0.0952
 3 EEM               0.00416 0.0616  0.0025  0.0269     0.0933
 4 VNQ               0.00664 0.0661  0.0025  0.0627     0.217
 5 TLT               0.00358 0.0416  0.0025  0.0258     0.0895
 6 IAU               0.00760 0.0488  0.0025  0.105      0.362
 7 HYG               0.00436 0.0307  0.0025  0.0605     0.210
 8 Equal_Weighted    0.00564 0.0355  0.0025  0.0884     0.306
 9 Naive_Risk_Parity 0.00555 0.0329  0.0025  0.0926     0.321
10 Minimum_Variance  0.00567 0.0207  0.0025  0.153      0.531
11 Tangency_Portfolio 0.0153 0.0415  0.0025  0.308      1.07
12 Risk_Free         0.0025  0       0.0025 NA         NA
```

Illustration – Efficient frontier (ETF)

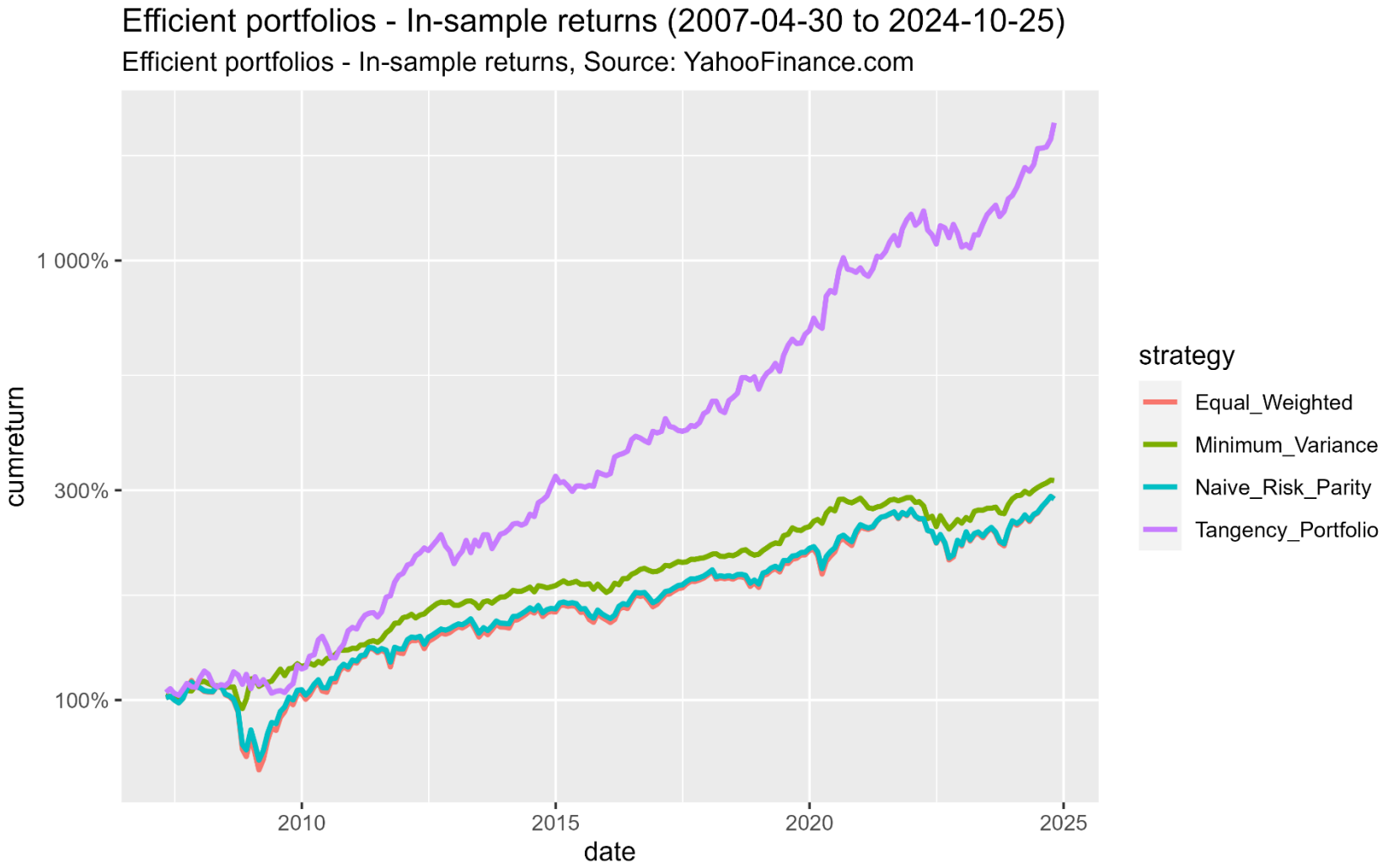
Efficient portfolios (2007-04-11 to 2024-10-25)

Efficient portfolios, Source: YahooFinance.com



- The compared portfolios are: (portfolios allow for shorts but no leverage: $\boldsymbol{w}'\mathbf{1} = 1$)
  - **Equal Weighted portfolio:** Heuristic portfolio that allocates $\boldsymbol{w} = \mathbf{1}/N$
  - **Naive Risk Parity:** Heuristic portfolio that allocates $\boldsymbol{w} \propto 1/\boldsymbol{\sigma}$
  - **Minimum variance:** Optimized minimum variance portfolio $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}\mathbf{1}$
  - **Tangency portfolio:** Optimized tangency portfolio (max. Sharpe) $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})$

Optimized ETF portfolios (In-Sample)



- Logarithmic plot was chosen as otherwise the lower curves are not so visible
- Tangency portfolio clearly outperforms all others, followed by the Minimum Variance portfolio
- **Note:** The results are **in-sample** → Future performance is likely to not be as good (!)

Extension 1.B: Out-sample test

- **Task:** Assume that wou have performed portfolio optimization based on 7 ETFs (listed below) at the end of the year 2019 and simulate what the performance of these portfolios would be in 2020-2024
- **List of ETFs:**
  - **SPY** = SPDR S&P 500 ETF Trust
  - **EFA** = iShares MSCI EAFE ETF
  - **EEM** = iShares MSCI Emerging Markets ETF
  - **VNQ** = Vanguard Real Estate Index Fund
  - **TLT** = iShares 20+ Year Treasury Bond ETF
  - **IAU** = iShares Gold Trust
  - **HYG** = iShares High Yield Corporate Bond ETF

MPT out-sample test (1)

Calculate μ and Σ on a restricted dataset that ends in 2019

```
# Extract return data - In-Sample
  sYear <- 2019
  mreturnsIn <- mdata %>%
    filter(year(date)<=sYear) %>%
    select(symbol,date,returnS) %>%
    spread(symbol,returnS) %>%
    select(date,symbols)
  # Convert to matrix - In-Sample
  retIn <- mreturnsIn %>%
    select(symbols)
  retIn <- as.matrix(retIn)
  # Compute parameters
  mju <- colMeans(retIn)
  covmat <- cov(retIn)
  cormat <- cor(retIn)
  sigma <- diag(covmat)^(1/2)
```

Using mju and covmat calculate the in-sample (pre-2019) portfolio weights:

```
> port_weights
# A tibble: 4 × 8
  strategy            SPY     EFA      EEM      VNQ   TLT     IAU     HYG
  <chr>             <dbl>   <dbl>    <dbl>    <dbl> <dbl>   <dbl>   <dbl>
1 Equal_Weighted    0.143   0.143   0.143     0.143   0.143 0.143   0.143
2 Naive_Risk_Parity 0.158   0.131   0.104     0.0990  0.168 0.130   0.210
3 Minimum_Variance  0.542  -0.110  -0.145    -0.224   0.317 0.0990  0.521
4 Tangency_Portfolio 1.32  -0.892   0.00208  -0.159   0.341 0.127   0.263
```

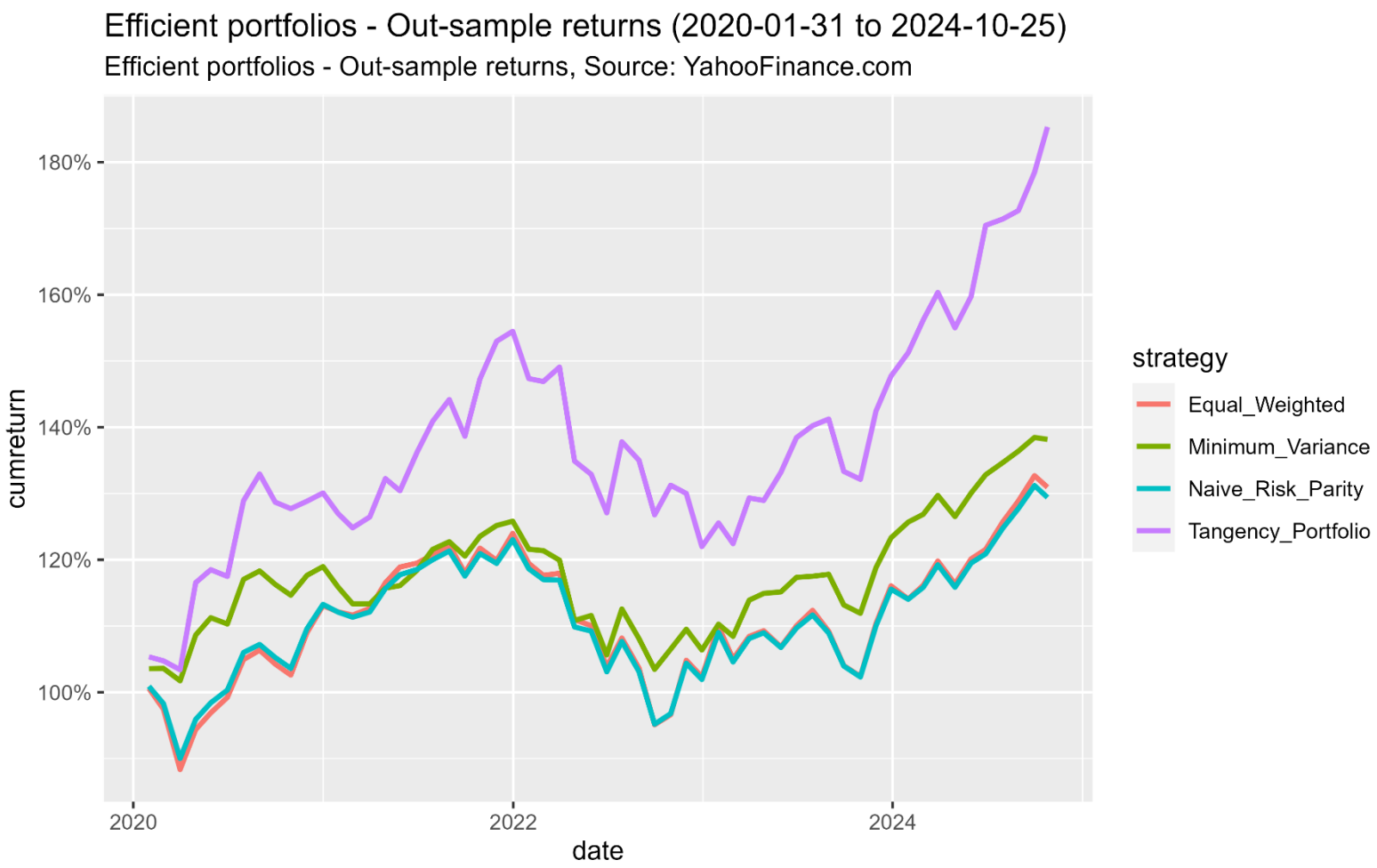
MPT out-sample test (2)

Plot the out-sample (post-2019) optimized portfolio returns:

```
# Plot cumulative returns - Out-Sample
dStart <- min(mdata$date[year(mdata$date)>sYear])
  dEnd <- max(mdata$date)
  gChart07 <- mreturns_long %>%
    filter(strategy %in% c_port) %>%
    filter(year(date)>sYear) %>%
    ungroup() %>%
    group_by(strategy) %>%
    mutate(cumreturn = cumprod(1+returnS)) %>%
    ggplot(aes(x = date, y = cumreturn, color = strategy)) +
    geom_line(size = 1) +
    labs(title = paste0("Efficient portfolios - Out-sample returns (",dStart," to ",dEnd,")"),
         subtitle = paste0("Efficient portfolios - Out-sample returns, Source:
YahooFinance.com")) +
    scale_y_continuous(labels = scales::percent)
  print(gChart07)
  ggsave(paste0(gFolder,"/gChart07.png"),gChart07,
         height = 5, width = 8, units = "in", dpi = 450)
```

Note: All of the inter-mediate steps are done identically as in the full-sample test

MPT Out-Sample test (2020-2024)



- The tangency portfolio estimated **in-sample** (i.e on pre-2019 data) works reasonably well **out-sample** (i.e. on the 2020-2024 time-period) → This is **not always the case (!)**
- Nevertheless, the outperformance is likely mostly caused by leveraged purchases of SPY
  → Problem of **overly concentrated portfolio weights**

Concentrated weights & leverage

- Weights of the **optimized portfolios** on the dataset from **2007-2019:**

	strategy	SPY	EFA	EEM	VNQ	TLT	IAU	HYG
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Equal_Weighted	0.143	0.143	0.143	0.143	0.143	0.143	0.143
2	Naive_Risk_Parity	0.158	0.131	0.104	0.0990	0.168	0.130	0.210
3	Minimum_Variance	0.542	-0.110	-0.145	-0.224	0.317	0.0990	0.521
4	Tangency_Portfolio	1.32	-0.892	0.00208	-0.159	0.341	0.127	0.263

- Weights of the **optimized portfolios** on the dataset from **2007-2024:**

	strategy	SPY	EFA	EEM	VNQ	TLT	IAU	HYG
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Equal_Weighted	0.143	0.143	0.143	0.143	0.143	0.143	0.143
2	Naive_Risk_Parity	0.147	0.130	0.108	0.101	0.160	0.137	0.217
3	Minimum_Variance	0.341	-0.0913	-0.117	-0.257	0.263	0.117	0.745
4	Tangency_Portfolio	1.96	-1.26	-0.127	-0.191	0.104	0.390	0.127

- In both cases the **Tangency portfolio** takes **large short positions** in some of the under-performing ETFs (EFA, VNQ) in order to finance a highly leveraged position in SPY ETF (+ some others) → Total **leverage** exceeds **2:1** in both tests
- While 2:1 leverage may still be acceptable, **unconstrained optimization** (with only the budget constraint) often results in much higher leverage (financed by large short positions in under-performing assets) → Need for **constrained optimization**
- **Legend: SPY** (SPDR S&P 500 ETF Trust), **TLT** (iShares 20+ Year Treasury Bond ETF), **VNQ** (Vanguard Real Estate Index Fund), **EFA** (iShares MSCI EAFE ETF), **EEM** (iShares MSCI Emerging Markets ETF), **IAU** (iShares Gold Trust), **HYG** (iShares High Yield Corporate Bond ETF)

Additional constraints

- For now we have considered only the **budget constraint:**
  - **Budget constraint:** $\boldsymbol{w'1} = 1$ (leverage is not allowed)
- In pratice it may be usefull to apply **additional/other constraints:**
  - **Long-only constraint:** $\boldsymbol{w} \geq 0$ (short positions not allowed)
  - **Holding constraint:** $\mathbf{l} \leq \boldsymbol{w} \leq \boldsymbol{u}$ ($\mathbf{l}$ and $\boldsymbol{u}$ set the min/max weights)
  - **Leverage constraint:** $\|\boldsymbol{w}\|_1 \leq L$ (limit on the total leverage)
  - **Cardinality constraint:** $\|\boldsymbol{w}\|_0 \leq N$ (we want to hold max. N assets)
  - **Turnover constraint:** $\|\boldsymbol{w} - \boldsymbol{w_0}\| \leq u$ ($\boldsymbol{w_0}$ is our current portfolio)
  - **Dollar-neutral constraint:** $\boldsymbol{w'1} = 0$ (value of short = value of long)
  - **Beta-neutral constraint:** $\boldsymbol{w'\beta} = 0$ ($\boldsymbol{\beta}$ = vector of betas)
  - **Market-neutral constraint:** $\boldsymbol{B'w} = \mathbf{0}$ ($\boldsymbol{B}$ = matrix of sensitivities)

    (assuming $\boldsymbol{r_t} = \boldsymbol{\alpha} + \boldsymbol{Bf_t} + \boldsymbol{\varepsilon_t}$)
- For most constraints (including Long-only) **no analytical solutions** exist:
  - Numerical solutions of **quadratic programing** problems (R package **quadprog**)
    - Zivot, E. (2021): https://bookdown.org/compfinezbook/introcompfinr/
  - Numerical solutions of **fractional programing** problems (bi-section method, etc.):
    - Palomar, D.P. (2020): https://palomar.home.ece.ust.hk/ELEC5470_lectures/slides_portfolio_optim.pdf
  - General purpose **constrained optimization algorithms** (R package **fmincon**)
  - Heuristic **global optimization algorithms** (simulated annealing, genetic algorithms, etc.)

Optimization algorithms in R

- **Optimization algorithms available in the R optim function:**
  - Base R offers a range of optimization algorithms in the **optim function**
  - The function syntax is: `optim(x0, objective, method = …)`
  - The **available optimizers** (methods) are:
  - **Local derivative-based unconstrained optimization:**
    - **BFGS** → Second-order method (extension of gradient descent with second derivatives), using an approximation of the Hessian (matrix of second derivatives)
    - **CG** → Conjugate Gradient method (another extension of gradient descent, which is less robust, but is suitable for more high-dimensional problems than BFGS)
  - **Local derivative-based constrained optimization:**
    - **L-BFGS-B** → Constrained version of the BFGS algorithm
  - **Local derivative-free unconstrained optimization:**
    - **Nelder-Mead** → Robust simplex-based heuristic local optimization method that is applicable to non-differentiable functions
  - **Global derivative-free unconstrained optimization:**
    - **SANN** → Simulated annealing (stochastic heuristic method to perform global optimization by randomly sampling new solutions and accepting them when a) f(x) is lower, or b) when f(x) is higher with probability that decreases with iterations
  - **Univariate hybrid-method local optimization**
    - **Brent** → Method based on the combination of **bisection**, **secant method** and **Newton's** method (exhibits the robustness of bisection with faster convergence)

Strong vs. weak constrains

- **Strong vs. weak constrains:**
  - For many solvers that do not allow **strong constrains**, we can at least apply **weak constrains** by **adjusting the objective function** $f(\boldsymbol{x})$
  - Example of **weak inequality constrains** $\boldsymbol{Bx} \leq \boldsymbol{b}$**:**
    - Replace $f(\boldsymbol{x})$ with $f^*(\boldsymbol{x}) = f(\boldsymbol{x}) + c\|(\boldsymbol{Bx} - \boldsymbol{b})I[(\boldsymbol{Bx} - \boldsymbol{b}) > 0]\|$
    - Where $I[(\boldsymbol{Bx} - \boldsymbol{b}) > 0]$ denotes an indicator function taking the value of 1 for the rows where the applied constrain is above zero
- **Indirect application of strong constrains:**
  - Another option of how to use constrains even for solvers that do not allow it is to replace the function $f(\boldsymbol{x})$ with a modified function $f^*(\boldsymbol{z})$ where $\boldsymbol{z} = g(\boldsymbol{x})$ is selected in such a way as to assure that optimization over $\boldsymbol{z}$ will always fulfill the constrains for $\boldsymbol{x}$
  - **Example 1:** Constrain that $\boldsymbol{x} > \boldsymbol{0}$ for all elements of $\boldsymbol{x}$**:**
    - We set $\boldsymbol{z} = \log(\boldsymbol{x})$ and run the optimization for $f^*(\boldsymbol{z})$ using that $\boldsymbol{x} = \exp(\boldsymbol{z})$
  - **Example 2:** Constrain that $\boldsymbol{x} < \boldsymbol{0}$ for all elements of $\boldsymbol{x}$**:**
    - We set $\boldsymbol{z} = \log(-\boldsymbol{x})$ and run the optimization for $f^*(\boldsymbol{z})$ using that $\boldsymbol{x} = -\exp(\boldsymbol{z})$
  - **Example 3:** Constrain that $\boldsymbol{0} > \boldsymbol{x} > \boldsymbol{1}$ for all elements of $\boldsymbol{x}$**:**
    - We set $\boldsymbol{z} = \log[\boldsymbol{x}/(1 - \boldsymbol{x})]$ and run the opt. for $f^*(\boldsymbol{z})$ using $\boldsymbol{x} = 1/(1 + e^{-\boldsymbol{z}})$

Package pracma → fmincon

❖ **Package pracma → fmincon optimization function**

- Function **fmincon** is based on the Matlab optimization algorithms
- Generally much more robust performance than function **optim**
- Allows for wide range of possible **constrains**
- **fmincon syntax:**

```
fmincon(x0, fn, gr = NULL, ..., method = "SQP",
        A = NULL, b = NULL, Aeq = NULL, beq = NULL,
        lb = NULL, ub = NULL, hin = NULL, heq = NULL,
        tol = 1e-06, maxfeval = 10000, maxiter = 5000)
```

- Where **x0** denotes the starting point and **fn** the minimized function
- The **constrains** are specified with:
  - **A, b** → Linear inequality constrains → $\boldsymbol{Ax \leq b}$
  - **Aeq, beq** → Linear equality constrains → $\boldsymbol{A_{eq}x = b_{eq}}$
  - **lb, ub** → Box constrains → $\boldsymbol{l_b \leq x \leq u_b}$
  - **hin** → Non-linear inequality const. → $\boldsymbol{h_{in}(x) \leq 0}$
  - **heq** → Non-linear equality const. → $\boldsymbol{h_{eq}(x) = 0}$

Extension 1.C: Long-only constraint

- **Task:** Repeat the previous tasks with one additional optimized portfolio
  → Tangency portfolio with long-only constrain
- **List of ETFs:**
  - **SPY** = SPDR S&P 500 ETF Trust
  - **EFA** = iShares MSCI EAFE ETF
  - **EEM** = iShares MSCI Emerging Markets ETF
  - **VNQ** = Vanguard Real Estate Index Fund
  - **TLT** = iShares 20+ Year Treasury Bond ETF
  - **IAU** = iShares Gold Trust
  - **HYG** = iShares High Yield Corporate Bond ETF

Long-only constraint (1)

Use fmincon to find the tangency portfolio under the long-only constraint

```
# Load pracma library
library(pracma)
# Sharpe ratio function
mysharpe = function(w,mju,covmat,rf) {
  mw <- as.vector(t(w) %*% mju)
  sw <- as.vector((t(w) %*% covmat %*% w)^(1/2))
  Sharpe <- (mw-rf)/sw
  return(Sharpe)
}
# Objective function
objective = function(w) {
  obj <- -mysharpe(w,mju,covmat,rf)
  return(obj)
}
# Set constrains - Maximum and minimum weights
wmin <- numeric(length(mju))
wmax <- numeric(length(mju))+1
# Set constrains - Sum of weights equals to one
Aeq = t(rep(1,length(mju)))
beq = 1
# Run optimization
opt <- fmincon(w_ew, objective, Aeq = Aeq, beq = beq, lb = wmin, ub = wmax)
# Save long-only tangency portfolio weights
w_tc <- opt$par
```

Note: The other parts of the code need to be adjusted so that they calculate all of the statistics and returns also for the **w_tc** portfolio

Long-only tangency portfolio

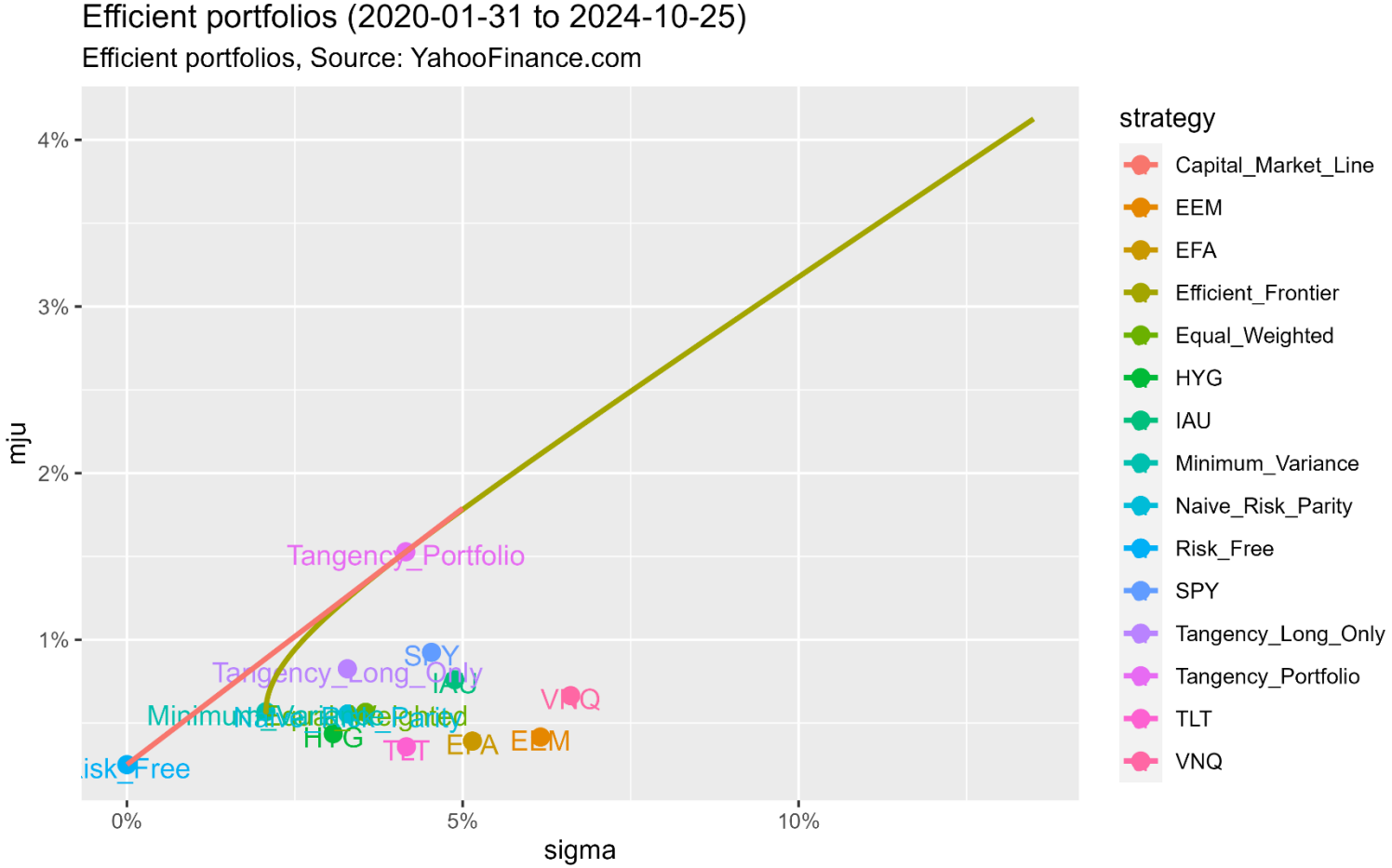
- **Weights** of the **optimized portfolios** on the dataset from **2007-2024:**

```
> port_weights
# A tibble: 5 × 8
  strategy              SPY      EFA      EEM      VNQ      TLT      IAU      HYG
  <chr>               <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 Equal_Weighted      0.143   0.143    0.143    0.143    0.143    0.143    0.143
2 Naive_Risk_Parity   0.147   0.130    0.108    0.101    0.160    0.137    0.217
3 Minimum_Variance    0.341  -0.0913  -0.117   -0.258    0.263    0.117    0.745
4 Tangency_Portfolio  1.96   -1.26    -0.127   -0.191    0.104    0.390    0.127
5 Tangency_Long_Only  0.588   0        0        0        0.0772   0.334    0
```

- **Risk-return characteristics** of all portfolios on the dataset from **2007-2024:**

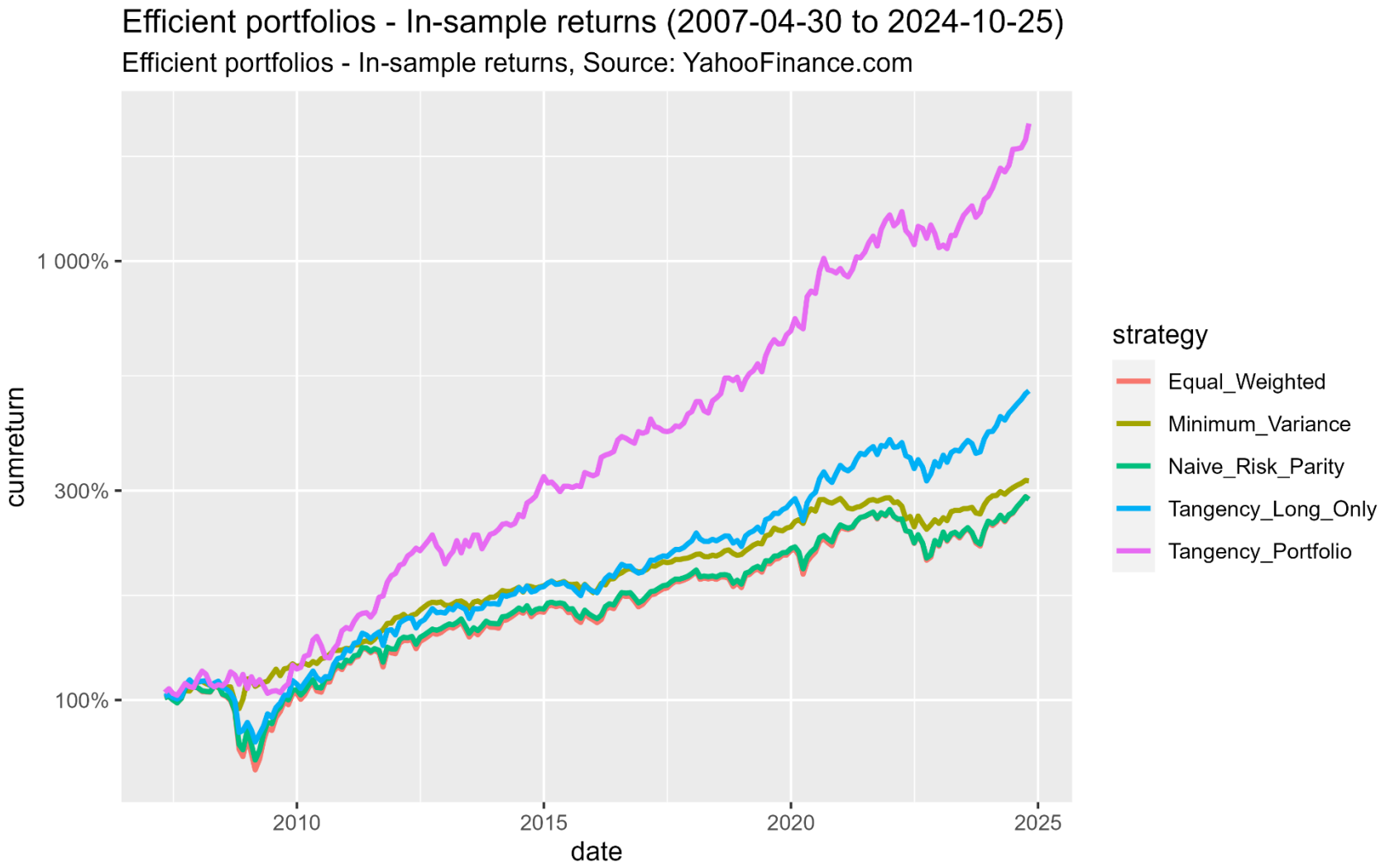
```
> port_char3
# A tibble: 13 × 6
   strategy              mju    sigma      rf   sharpe sharpe_pa
   <chr>               <dbl>    <dbl>   <dbl>    <dbl>     <dbl>
 1 SPY                 0.00924 0.0453  0.0025   0.149     0.515
 2 EFA                 0.00391 0.0514  0.0025   0.0275    0.0952
 3 EEM                 0.00416 0.0616  0.0025   0.0269    0.0933
 4 VNQ                 0.00664 0.0661  0.0025   0.0627    0.217
 5 TLT                 0.00358 0.0416  0.0025   0.0258    0.0895
 6 IAU                 0.00760 0.0488  0.0025   0.105     0.362
 7 HYG                 0.00436 0.0307  0.0025   0.0605    0.210
 8 Equal_Weighted      0.00564 0.0355  0.0025   0.0884    0.306
 9 Naive_Risk_Parity   0.00555 0.0329  0.0025   0.0926    0.321
10 Minimum_Variance    0.00567 0.0207  0.0025   0.153     0.531
11 Tangency_Portfolio  0.0153  0.0415  0.0025   0.308     1.07
12 Tangency_Long_Only  0.00826 0.0328  0.0025   0.175     0.608
13 Risk_Free           0.0025  0       0.0025   NA        NA
```

Illustration – Efficient frontier (ETF)



- The compared portfolios are: (portfolios allow for shorts but no leverage: $\boldsymbol{w}'\mathbf{1} = 1$)
  - **Equal Weighted portfolio:** Heuristic portfolio that allocates $\boldsymbol{w} = \mathbf{1}/N$
  - **Naive Risk Parity:** Heuristic portfolio that allocates $\boldsymbol{w} \propto 1/\boldsymbol{\sigma}$
  - **Minimum variance:** Optimized minimum variance portfolio $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}\mathbf{1}$
  - **Tangency portfolio:** Optimized tangency portfolio (max. Sharpe) $\boldsymbol{w} \propto \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu} - r\mathbf{1})$

Optimized ETF portfolios (In-Sample)



- Logarithmic plot was chosen as otherwise the lower curves are not so visible
- Long-only tangency portfolio still shows a **decent performance**
- **Note:** The results are **in-sample** → Future performance is likely to not be as good (!)

Long-only tangency portfolio (IS vs. FS)

- **Weights** of the **optimized portfolios** on the dataset from **2007-2019:**

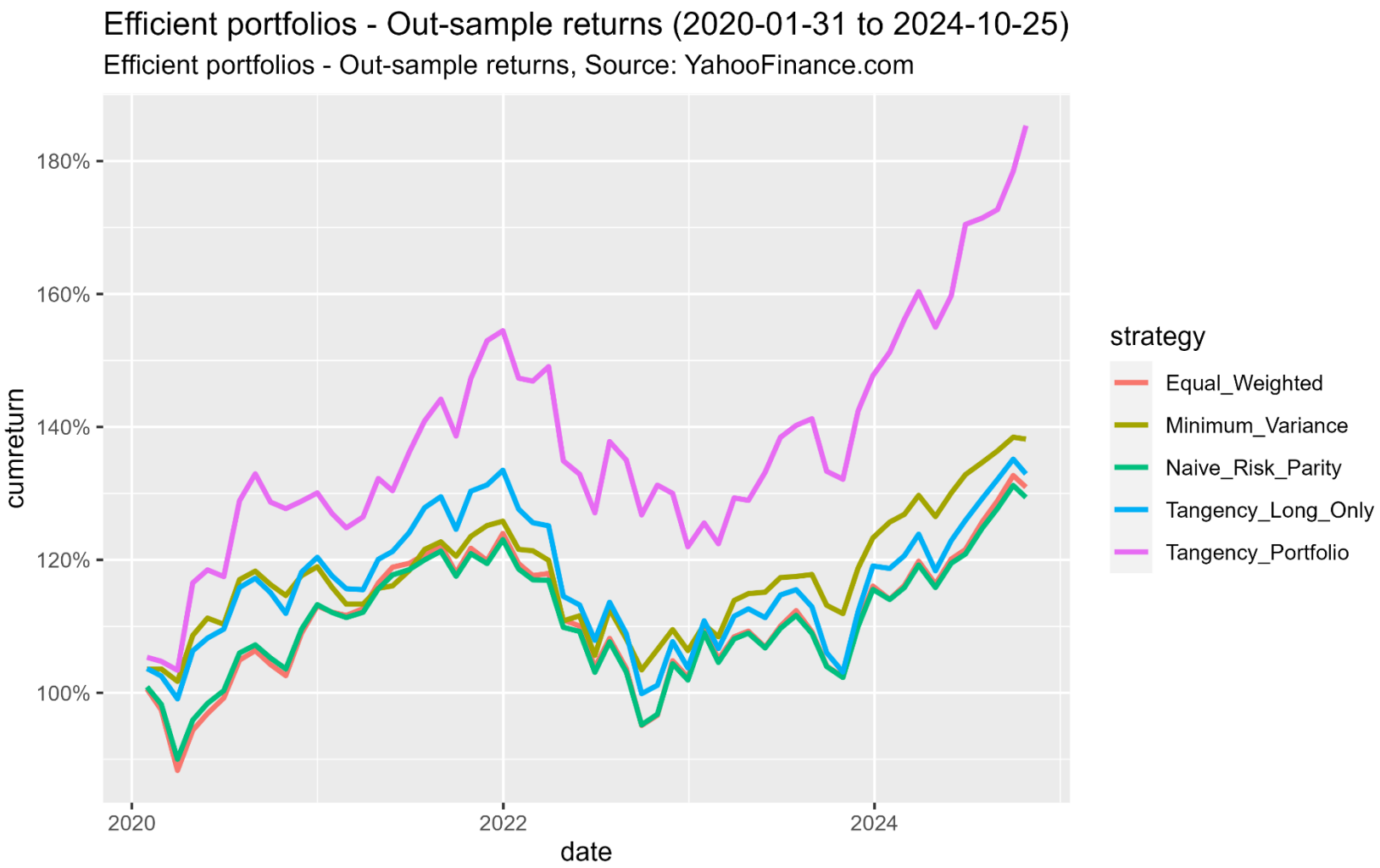
```
> port_weights
# A tibble: 5 × 8
  strategy             SPY     EFA     EEM     VNQ   TLT     IAU     HYG
  <chr>              <dbl>   <dbl>   <dbl>   <dbl> <dbl>   <dbl>   <dbl>
1 Equal_Weighted     0.143   0.143   0.143   0.143 0.143   0.143   0.143
2 Naive_Risk_Parity  0.158   0.131   0.104   0.099 0.168   0.130   0.210
3 Minimum_Variance   0.542  -0.11   -0.145  -0.224 0.317   0.099   0.521
4 Tangency_Portfolio 1.32   -0.892   0.0021 -0.159 0.341   0.127   0.263
5 Tangency_Long_Only 0.487  0       0       0     0.426   0.0868  0
```

- **Weights** of the **optimized portfolios** on the dataset from **2007-2024:**

```
> port_weights
# A tibble: 5 × 8
  strategy             SPY     EFA     EEM     VNQ     TLT   IAU     HYG
  <chr>              <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>   <dbl>
1 Equal_Weighted     0.143   0.143   0.143   0.143   0.143 0.143   0.143
2 Naive_Risk_Parity  0.147   0.130   0.108   0.101   0.160 0.137   0.217
3 Minimum_Variance   0.341  -0.0913 -0.117  -0.258   0.263 0.117   0.745
4 Tangency_Portfolio 1.96   -1.26   -0.127  -0.191   0.104 0.390   0.127
5 Tangency_Long_Only 0.588  0       0       0       0.0772 0.334   0
```

- We can see that on the full-sample optimization (2007-2024) results in much **lower weights** of **TLT** (US gov. bonds) and **higher weights** of **IAU** (gold)
- We could not know in advance (in 2019) that TLT would under-perform in 2020-2024 and IAU will out-perform

MPT Out-Sample test (2020-2024)



- **Out-sample performance** of the **long-only tangency portfolio** is **poor**
- This is most likely caused by its **high weight** of **TLT** which **performed poorly** in **2020-2024**

Portfolio optimization and CAPM

- Modern Portfolio Theory (MPT)
- Portfolio optimization in practice
- Case study 1 – MPT application on ETFs
- **Case study 2A – MPT application on stocks**
- Case study 2B – MPT application on stocks with CAPM

Case study 2: MPT on stocks

- **Task:** Download the data for all DJIA stocks in the period since 2011. Using the data from 2011-2018, find the optimal (maximum Sharpe ratio) portfolio weights (long-only), and compare the returns of the optimized portfolio in years 2019-2024 with the returns of an equally weighted benchmark.
- Estimated $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ from **(A) historical returns** and the **(B) CAPM model**
- **Steps – Variant A:**
  1. Download OHLCV data for the DJIA stocks in the period from 2011-2024
  2. Compute daily logarithmic returns of all DJIA stocks
  3. Compute SPY log-returns and join them to the DJIA dataset (used in **Variant B** only)
  4. Compute the monthly returns for all stocks and the index
  5. Convert the monthly returns in period 2011-2018 into a matrix
  6. Compute the mean vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$
  7. Find equally-weighted and unconstrained-tangency portfolios (as benchmark)
  8. Prepare function to compute $Sharpe_{\boldsymbol{w}} = \frac{\mu_{\boldsymbol{w}} - r_f}{\sigma_{\boldsymbol{w}}}$ given weight vector $\boldsymbol{w}$, by using the relationship that $\mu_{\boldsymbol{w}} = \boldsymbol{w}'\boldsymbol{\mu}$ and $\sigma_{\boldsymbol{w}} = \sqrt{\boldsymbol{w}'\boldsymbol{\Sigma}\boldsymbol{w}}$
  9. Run optimization to find $\widehat{\boldsymbol{w}} = \max_{\boldsymbol{w}} \frac{\mu_{\boldsymbol{w}} - r_f}{\sigma_{\boldsymbol{w}}}$ with constrains $\sum \boldsymbol{w} = 1$ and $\boldsymbol{w} > 0$
  10. Compute returns for the optimized portfolio in the period from 2019-2021

Portfolio optimization (1)

1. Download OHLCV data for the DJIA stocks in the period from 2011-2021

```
# Load tidyquant
library(tidyverse)
library(tidyquant)
library(broom)
library(pracma)
# Get DJIA components
info <- tq_index("DOW")
# Settings
tStart <- "2011-01-01"
tEnd   <- today()
# Remove erronous row
info <- filter(info, company != "US DOLLAR")
# Load DJIA stock prices (+ include info about each stock)
prices <- info %>% tq_get(get = "stock.prices", from = tStart, to = tEnd)
```

Additional R packages:

- **pracma** constains general purpose optimization functions which will be used to find the long-only constrained tangency portfolio
- **broom** allows us to easily compute regressions for grouped variables which will be used to estimate CAPM for individual stocks (in variant B)

Precausory step: Remove stocks that were not traded since the start of 2011

```
# Remove stocks that were not available since the first day of the sample
minDate <- min(prices$date)
prices2 <- prices %>%
  group_by(symbol) %>%
  mutate(mindate = min(date)) %>%
  filter(mindate==minDate)
```

Note: Among the current (2024/10) components of DJIA all stocks have existed since 2011 (making this step redundant on the current dataset)

Portfolio optimization (2)

2. Compute daily logarithmic returns of all DJIA stocks

```
# Compute daily log-returns
prices2 <- prices2 %>%
  group_by(symbol) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return))
```

3. Compute SPY logarithmic returns and join them to the DJIA dataset

```
# Download SPY (market benchmark)
index <- tq_get("SPY",get = "stock.prices", from = tStart, to = tEnd)
# Index daily log-returns
index <- index %>%
  group_by(symbol) %>%
  mutate(logprice = log(adjusted),
         return = logprice - lag(logprice),
         return = ifelse(is.na(return),0,return))
# Select stock index returns
temp <- index %>%
  ungroup() %>%
  mutate(mreturn = return) %>%
  select(date,mreturn)
# Join index returns to the price data
prices3 <- prices2 %>%
  left_join(temp,by="date")
```

Note: SPY (S&P500) ETF will be used as proxy for the overal market in the estimation of the CAPM model (Variant B)

Portfolio optimization (3)

4. Compute the monthly returns for all stocks and the index

```
# Convert to monthly frequency
prices4 <- prices3 %>%
  mutate(year = year(date),
         month = month(date)) %>%
  group_by(symbol,year,month) %>%
  summarize(date = max(date),
            return = sum(return),
            mreturn = sum(mreturn)) %>%
  ungroup() %>%
  mutate(returnS = exp(return)-1,
         mreturnS = exp(mreturn)-1)
# Extract log-returns
logret <- prices4 %>%
  select(date,symbol,return,mreturn)
# Extract simple returns
simret <- prices4 %>%
  select(date,symbol,returnS,mreturnS) %>%
  rename(return = returnS,
         mreturn = mreturnS)
```

Note: When aggregating returns from daily to monthly, we should use log-returns. We can then convert the monthly log-returns to simple returns with **exp(return)-1**

Note: Portfolio optimization will be run on **simple returns** which are more adequate for the aggregation of stocks into a portfolio
(the code is prepared in way to allow switching between simple and log-returns if needed)

Portfolio optimization (4)

5. Convert the monthly returns in period 2011-2018 into a matrix

```
# In-sample cut-off
sYear <- 2018
# Convert returns to a wide structure
wlogret <- logret %>%
  spread(symbol,return)
wsimret <- simret %>%
  spread(symbol,return)
# Select in-sample returns
wlogretIn <- filter(wlogret, year(date) <= sYear)
wsimretIn <- filter(wsimret, year(date) <= sYear)
# Convert logreturns into matrices
XLF <- as.matrix(wlogret[,3:ncol(wlogret)])
XL <- as.matrix(wlogretIn[,3:ncol(wlogretIn)])
# Convert simple returns into matrices
XSF <- as.matrix(wsimret[,3:ncol(wsimret)])
XS <- as.matrix(wsimretIn[,3:ncol(wsimretIn)])
# Alternative 1: Run optimization based on simple returns
XF <- XSF
X <- XS
ret <- simret
# Alternative 2: Run optimization based on log-returns
#XF <- XLF
#X <- XL
#ret <- logret
```

Portfolio optimization (5)

6. Compute the mean vector μ and covariance matrix Σ

```
# Set vector of expected returns as equal to the historic returns
mju <- colMeans(X)
# Compute historical correlation and covariance matrix
cormat <- cor(X)   # Correlation matrix
covmat <- cov(X)   # Covariance matrix
```

Note: Setting of vector μ as equal to the average historical returns represents major simplification, which may not always be suitable when working with individual stocks (i.e. past returns may not predict future returns), in Variant B, we will use the CAPM model to estimate the vector of expected returns instead.

7. Find benchmark portfolio weights

```
# Equaly weighted portfolio weights
w_ew <- rep(1/nrow(S),nrow(S))
names(w_ew) <- names(mju)
# Unconstrained tangency portfolio weights
w_ta <- (ginv(covmat) %*% (as.vector(mju)-rf)) /
        sum(ginv(covmat) %*% (as.vector(mju)-rf))
w_ta <- as.vector(w_ta)
names(w_ta) <- names(mju)
```

Note: While unconstrained tangency portfolio can be found analytically (with the formula above), for the constrained (long-only) portfolio we need to use numerical optimization as shown on the next slide.

Portfolio optimization (6)

8. Prepare the objective function for the optimization

```
# Sharpe ratio function
mysharpe = function(w,mju,covmat,rf) {
  mw <- as.vector(t(w) %*% mju)
  sw <- as.vector((t(w) %*% covmat %*% w)^(1/2))
  Sharpe <- (mw-rf)/sw
  return(Sharpe)
}
# Objective function
objective = function(w) {
  obj <- -mysharpe(w,mju,covmat,rf)
  return(obj)
}
```

Note: As the standard solvers are set for the minimization task, we need to use negative value of the Sharpe ratio as the objective function

9. Run optimization to find the long-only tangency portfolio weights

```
# Weight bounds (LONG ONLY)
wmin <- numeric(length(w_ew))
wmax <- numeric(length(w_ew))+1
# Set constrains - Sum of weights equals to one
Aeq = t(rep(1,length(w_ew)))
beq = 1
# Run optimization
opt <- fmincon(w_ew, objective, Aeq = Aeq, beq = beq, lb = wmin, ub = wmax)
# Save long-only tangency portfolio weights
w_tc <- opt$par
names(w_tc) <- names(mju)
```

Portfolio optimization (5)

10. Compute returns of the optimized portfolios in the period from 2019-2021

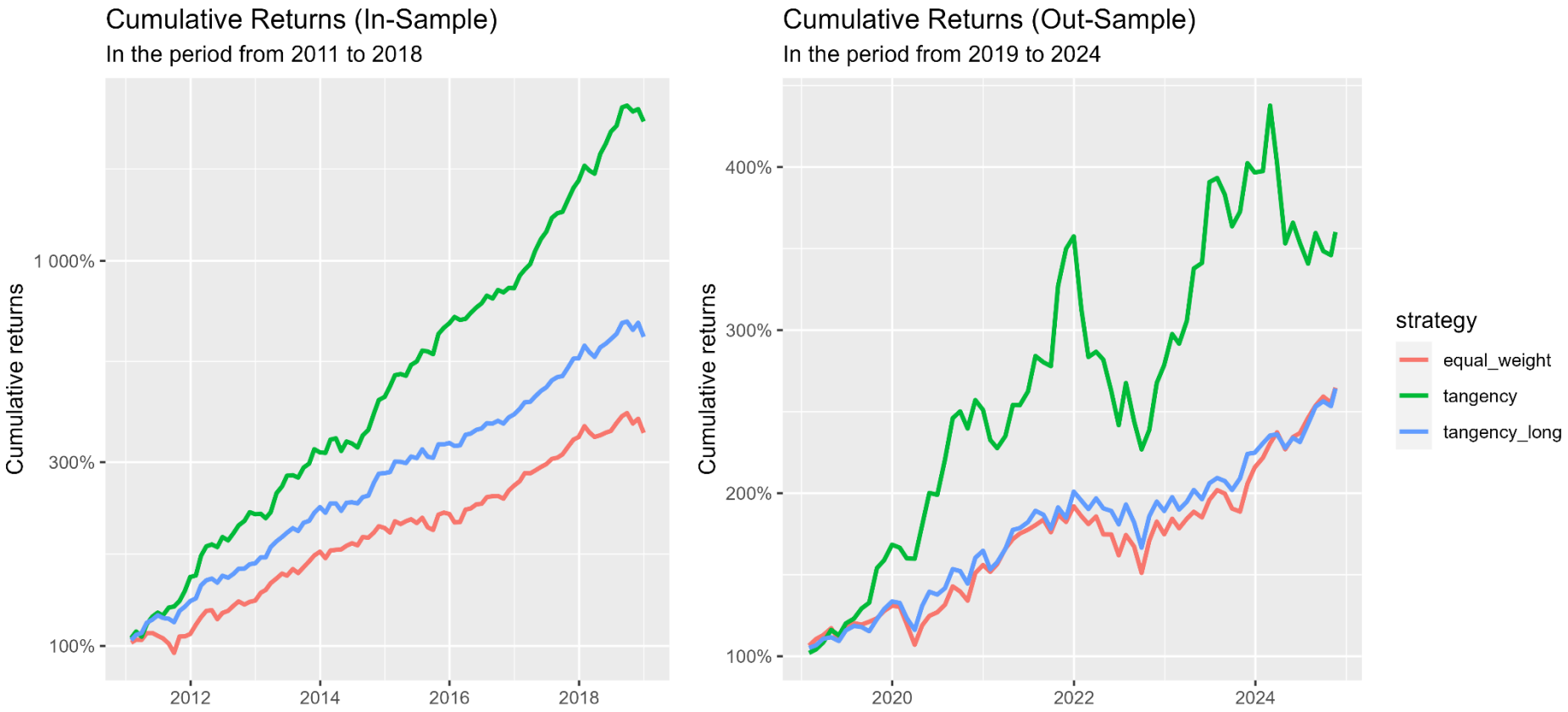
```
# Compute portfolio returns (full-sample)
R_EW <- XSF %*% w_ew # Equally weighted benchmark
R_TA <- XSF %*% w_ta # Tangency portfolio (unconstrained)
R_TC <- XSF %*% w_tc # Tangency portfolio (constrained)
# Join into a tibble
portfolio <- tibble("date" = wsimret$date,
                    "equal_weight" = R_EW,
                    "tangency" = R_TA,
                    "tangency_long" = R_TC)
# Gather
portfolio <- portfolio %>%
  gather(strategy,return,-date)
# Plot on chart - Out-sample
c02 <-portfolio %>%
  filter(year(date)>sYear) %>%
  group_by(strategy) %>%
  mutate(cumreturn = cumprod(1+return)) %>%
  ggplot(aes(x = date, y = cumreturn, color = strategy)) +
  geom_line(size = 1) +
  labs(title = "Cumulative Returns (Out-Sample)",
       subtitle = paste0("In the period from ",sYear," to ",year(tEnd)),
       x = "", y = "Cumulative returns") +
  scale_y_continuous(labels = scales::percent)
print(c02)
```

Note: To compute the in-sample returns you can just apply the opposite filter:

```
filter(year(date)<=sYear)
```

Portfolio optimization – Results

Full-Sample (2011-2021) vs. Out-Sample (2019-2023) returns:



Tangency (Long-Only): We can see that in the in-sample period (2011-2018), the optimized portfolio significantly outperforms the equally-weighted benchmark, in the out-sample (2019-2021) it outperforms it only slightly

Tangency (Unconstrained): The unconstrained tangency portfolio outperforms in both periods (based on returns), but at the cost of much higher risk and leverage, i.e. **sum(abs(w_ta))=5.06**

Portfolio optimization and CAPM

- Modern Portfolio Theory (MPT)
- Portfolio optimization in practice
- Case study 1 – MPT application on ETFs
- Case study 2A – MPT application on stocks
- **Case study 2B – MPT application on stocks with CAPM**

Portfolio optimization & CAPM

- **Task:** Download the data for all DJIA stocks in the period since 2011. Using the data from 2011-2018, find the optimal (maximum Sharpe ratio) portfolio weights (long-only), and compare the returns of the optimized portfolio in years 2019-2024 with the returns of an equally weighted benchmark.
- **Note:** To compute $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$, use the **CAPM model**
- **Steps – Variant B (replacing the calculations $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$):**
  1. Compute the CAPM betas on the in-sample part of the dataset (2011-2018):
     $$R_{i,t} - R_f = \alpha_i + \beta_i\left[R_{M,t} - R_f\right] + \epsilon_{i,t}$$
  2. Compute vector of CAPM expected stock returns $\boldsymbol{\mu} = [\mathrm{E}(R_1), \dots, \mathrm{E}(R_n)]$:
     $$\mathrm{E}(R_i) = R_f + \beta_i\left[\mathrm{E}(R_M) - R_f\right]$$
  3. Compute the CAPM model residual returns on 2011-2018:
     $$\epsilon_{i,t} = R_{i,t} - \alpha_i - \beta_i\left[R_{M,t} - R_f\right] - R_f$$
  4. Compute covariance matrix of residual returns $\boldsymbol{\Sigma}_{\epsilon}$
  5. Compute CAPM-implied covariance matrix $\boldsymbol{\Sigma} = \sigma_M^2 \boldsymbol{\beta}\boldsymbol{\beta}' + \boldsymbol{D}_{\boldsymbol{\epsilon}}$
  6. Run portfolio optimization using the CAPM implied $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$
  7. Compute returns for the optimized portfolio in the period from 2019-2021
- **Note:** For simplicity assume $R_f = 1\%$ p.a. and $\mathrm{E}(R_M) = 10\%$ p.a.

Case study: Portfolio optimization

1. Compute the CAPM betas on the in-sample part of the dataset (2011-2018):

```
# Set expected market return and risk-free rate
mjuM <- 0.1/12 # Expected market return
rf <- 0.01/12  # Risk-free return
# Compute CAPM regressions (on in-sample)
fit <- ret %>%
  filter(year(date) <= sYear) %>%
  select(symbol,date,return,mreturn) %>%
  mutate(return = return-rf,
         mreturn = mreturn-rf) %>%
  nest(data = -symbol) %>%
  mutate(
    fit = map(data, ~ lm(return ~ mreturn, data = .x)),
    tidied = map(fit, tidy)
  )
# Extract results
fit <- fit %>%
  select(symbol,tidied) %>%
  unnest(tidied)
# Save CAPM betas
CAPM <- fit %>%
  select(symbol,term,estimate) %>%
  spread(term,estimate)
names(CAPM) <- c("symbol","alpha","beta")
```

Note: We have used the package **broom** to compute the regression:

$$R_{i,t} - R_f = \alpha_i + \beta_i\left[R_{M,t} - R_f\right] + \epsilon_{i,t}$$

For each stock i in the DJIA index and get the vector of betas $\boldsymbol{\beta} = [\beta_1, \dots, \beta_n]$

Illustration CAPM betas

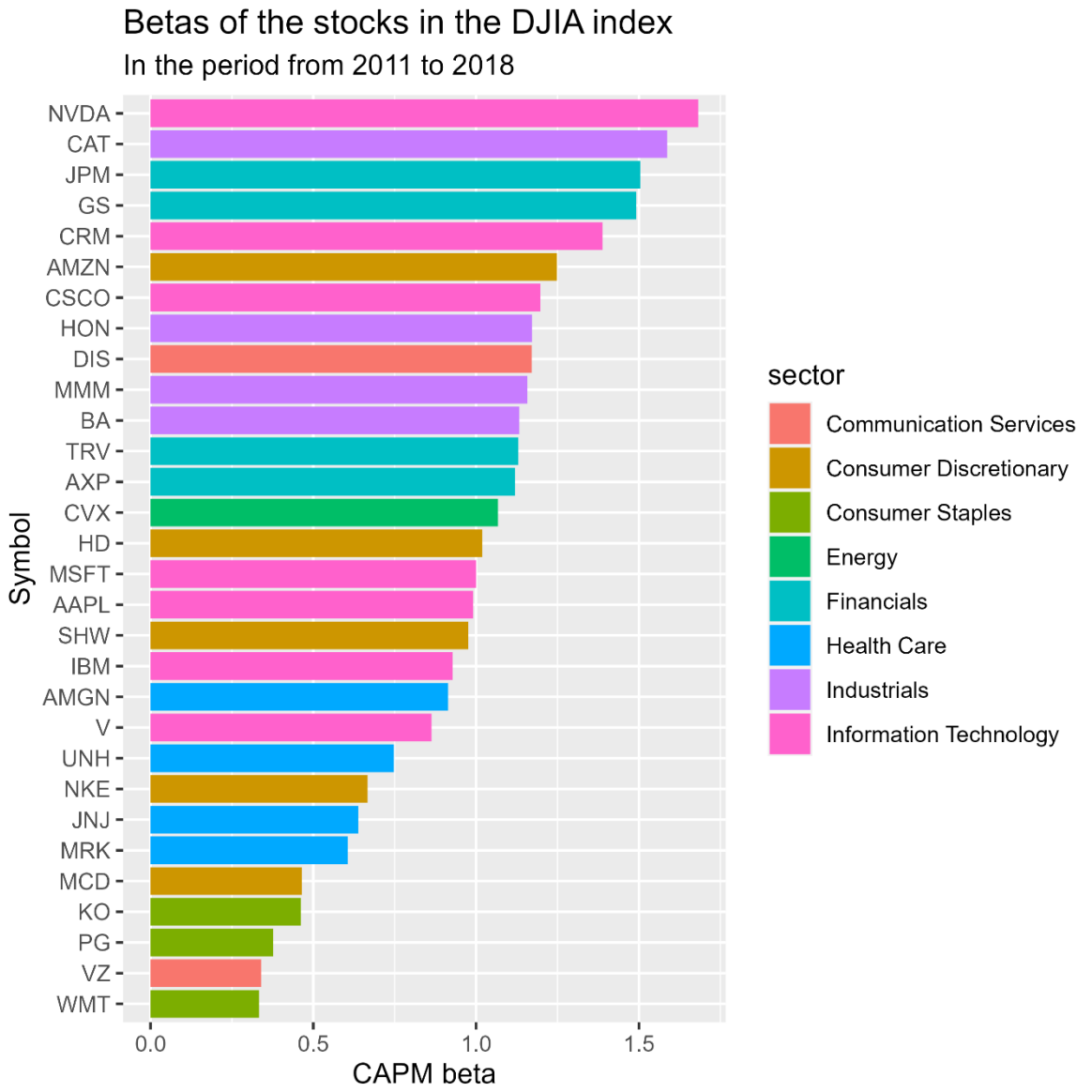
Illustration – Figure showing CAPM betas for all stocks (with sector information)

```
# Illustration - CAPM Betas
  temp <- info %>%
    select(symbol,sector)
  c93 <- CAPM %>%
    left_join(temp,by="symbol") %>%
    ggplot(aes(x = reorder(symbol,beta), y = beta, fill = sector)) +
    geom_col() +
    labs(title = paste0("Betas of the stocks in the DJIA index"),
         subtitle = paste0("In the period from ",year(tStart)," to ",sYear),
         y = "CAPM beta", x = "Symbol", color = "") +
    coord_flip()
  print(c93)
```

Note: Since the latest Tidyquant update, the function **tq_index("DOW")** does not provide the sector information (the column „sector“ is empty), in order to include sector information in the chart, we have thus exported the **info** table to Excel, filled-in the individual stock sectors manualy and imported it back to R:

```
# Export to xlsx (to manually fill the sectors)
write_xlsx(info,"DJIA_Info_2024.xlsx") # Export to Excel
info <- read_xlsx("DJIA_Info_2024.xlsx") # Import from Excel
```

Illustration – CAPM Betas



Highest betas are observed for:

- **Financials**
- **Industrials**
- **Information Technology**

Lowest betas are observed for:

- **Consumer Stables**
- **Communication services**
- **Health Care**

Case study: Portfolio optimization

2. Compute vector of CAPM expected stock returns $\boldsymbol{\mu}$:

```
# Compute CAPM expected returns
CAPM <- CAPM %>%
  mutate(mju =  rf + (mjuM - rf)*beta)
# Extract CAPM betas
betas <- CAPM$beta
names(betas) <- CAPM$symbol
# Extract daily expected returns
M_CAPM <- CAPM$mju
names(M_CAPM) <- CAPM$symbol
```

Note: CAPM expected returns:

$$\mathrm{E}(R_i) = R_f + \beta_i\left[\mathrm{E}(R_M) - R_f\right]$$

We further extract the vectors of betas ($\boldsymbol{\beta}$) and expected returns ($\boldsymbol{\mu}$) for use in later parts of the code:

$$\boldsymbol{\mu} = [\mathrm{E}(R_1), \dots, \mathrm{E}(R_n)]$$
$$\boldsymbol{\beta} = [\beta_1, \dots, \beta_n]$$

3. Compute the CAPM model residual returns on 2011-2018:

```
# Compute residual returns
resretIn <- ret %>%
  filter(year(date) <= sYear) %>%
  left_join(CAPM,by="symbol") %>%
  mutate(pred = alpha + beta*(mreturn - rf)+rf,
         res = return - pred)
mean(resretIn$res)
# Convert to a wide format
wresretIn <- resretIn %>%
  select(date, symbol, res, mreturn) %>%
  spread(symbol,res)
```

Note: To compute the CAPM implied covariance matrix $\boldsymbol{\Sigma}$, we need to first compute the CAPM residual returns:

$$\epsilon_{i,t} = R_{i,t} - \alpha_i - \beta_i\left[R_{M,t} - R_f\right] - R_f$$

Residual returns tell us the part of the individual stock variability that was not explained by CAPM regression

Case study: Portfolio optimization

4. Compute covariance matrix of residual returns Σ_ϵ

```
# Convert residual returns into matrix
XR <- as.matrix(wresretIn[,3:ncol(wresretIn)])
# Compute correlation and covariance matrix of residuals
R_Res <- cor(XR)   # Correlation matrix
S_Res <- cov(XR)   # Covariance matrix
```

Illustration – Correlation matrix of residual returns (first 7 stocks)

```
> round(R_Res[1:7,1:7],4)
```

	AAPL	AMGN	AMZN	AXP	BA	CAT	CRM
AAPL	1.0000	-0.0810	0.0930	-0.0986	-0.0666	-0.2061	0.1804
AMGN	-0.0810	1.0000	0.0830	0.0117	0.0766	-0.1544	-0.0071
AMZN	0.0930	0.0830	1.0000	-0.0234	0.1394	-0.1694	0.2342
AXP	-0.0986	0.0117	-0.0234	1.0000	0.1052	0.0581	0.0991
BA	-0.0666	0.0766	0.1394	0.1052	1.0000	-0.0402	-0.1853
CAT	-0.2061	-0.1544	-0.1694	0.0581	-0.0402	1.0000	0.0833
CRM	0.1804	-0.0071	0.2342	0.0991	-0.1853	0.0833	1.0000

Note: Correlations between CAPM residuals are low and often negative

Illustration – Correlation matrix of the raw returns (first 7 stocks)

```
> round(R[1:7,1:7],4)
```

	AAPL	AMGN	AMZN	AXP	BA	CAT	CRM
AAPL	1.0000	0.1539	0.2930	0.2202	0.2090	0.1483	0.3825
AMGN	0.1539	1.0000	0.3146	0.3357	0.3479	0.2283	0.2800
AMZN	0.2930	0.3146	1.0000	0.3186	0.3963	0.2248	0.4555
AXP	0.2202	0.3357	0.3186	1.0000	0.4528	0.4675	0.4380
BA	0.2090	0.3479	0.3963	0.4528	1.0000	0.3655	0.2153
CAT	0.1483	0.2283	0.2248	0.4675	0.3655	1.0000	0.4290
CRM	0.3825	0.2800	0.4555	0.4380	0.2153	0.4290	1.0000

Note: Correlations between stock returns are higher and mostly positive

Case study: Portfolio optimization

5. Compute CAPM-implied covariance matrix

```
# Volatility of market returns
sigmaM <- sd(wresretIn$mreturn)
# CAPM implied covariance matrix of returns
S_CAPM <- (sigmaM^2) * (betas %*% t(betas)) + diag(diag(S_Res))
```

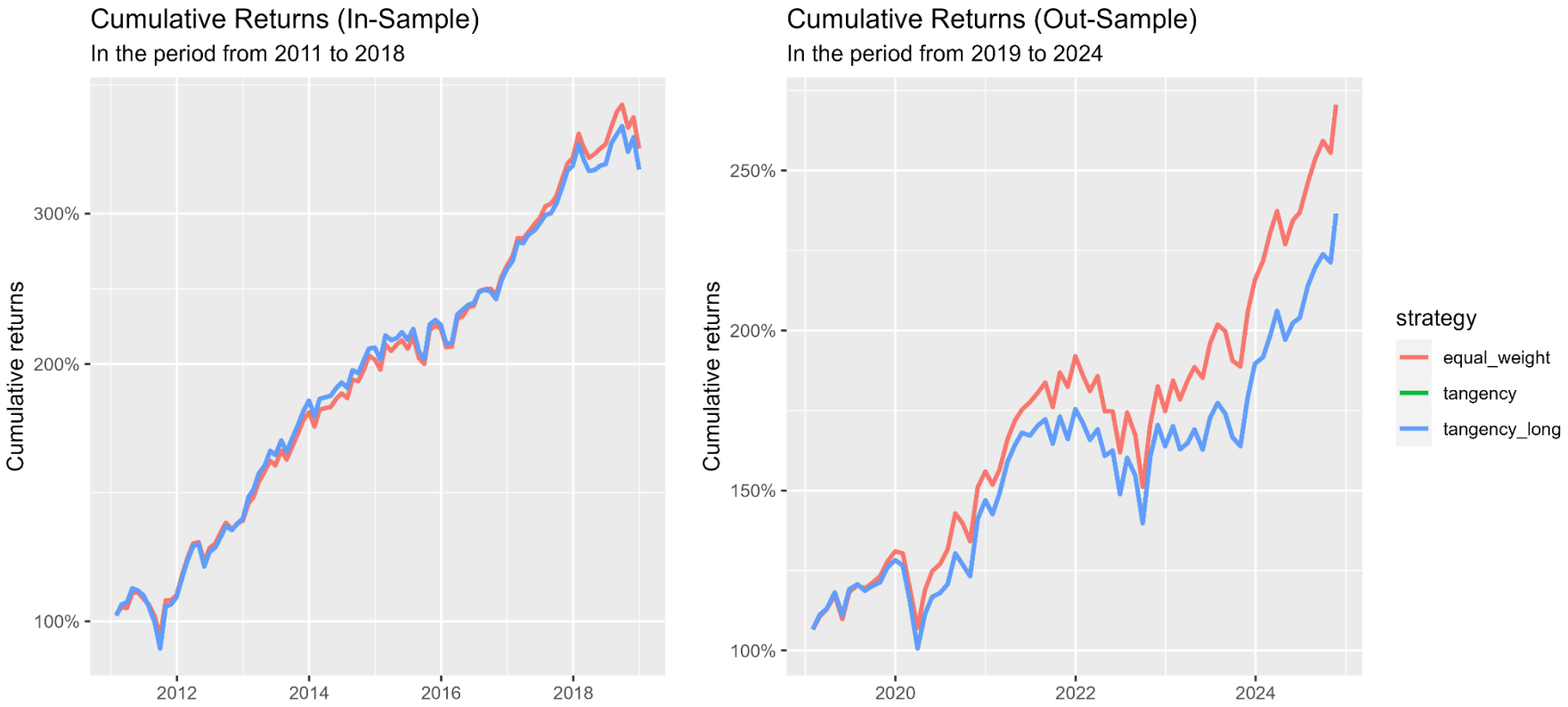
Note: CAPM implied covariance matrix is computed as $\boldsymbol{\Sigma} = \sigma_M^2 \boldsymbol{\beta}\boldsymbol{\beta}' + \boldsymbol{D}_{\boldsymbol{\epsilon}}$, where $\boldsymbol{\beta}$ is a vector of betas, σ_M^2 is the variance of market returns, and $\boldsymbol{D}_{\boldsymbol{\epsilon}}$is a diagonal matrix, containing the residual variances $\sigma_{\epsilon_i}^2$ on the diagonal and zeros elsewhere (i.e. the diagonal elements of $\boldsymbol{D}_{\boldsymbol{\epsilon}}$ are the same as in $\boldsymbol{\Sigma}_{\boldsymbol{\epsilon}}$, but the non-diagonal elements are replaced with zeroes) → Assumption of CAPM that ϵ_i and ϵ_j are uncorrelated between any two stocks → Acts as form of regularization of the covariance matrix

6. Run portfolio optimization using the CAPM implied $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$

(performed in the same way as in Case Study 2A, but with historical $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ replaced with the CAPM implied $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$)

CAPM Portfolio Opt.– Long-Only

Full-Sample (2011-2021) vs. Out-Sample (2019-2021) returns:



Optimization did not led to improvement (compared to equal weighted portfolio) even on the in-sample period → CAPM assumes simple correlation structure (i.e. all correlations between stocks are explained by betas) + expected returns depend only on betas → Little room for optimization

Possibly more suitable approach → Use CAPM to regularize the historical estimates of $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$ → $\boldsymbol{\mu} = (1 - c_{\boldsymbol{\mu}})\boldsymbol{\mu}_{hist} + c_{\boldsymbol{\mu}}\boldsymbol{\mu}_{CAPM}$ and $\boldsymbol{\Sigma} = (1 - c_{\boldsymbol{\Sigma}})\boldsymbol{\Sigma}_{hist} + c_{\boldsymbol{\Sigma}}\boldsymbol{\Sigma}_{CAPM}$

Check of the CAPM (1)

- **Task:** Does the CAPM predictive equation even work? Compute linear regression between the (i) in-sample (2011-2018) and (ii) out-sample (2019-2024) average monthly stock returns and the in-sample estimates of CAPM expected returns
- **In-sample check:**

```
# Historical returns (p.a.) (in-sample)
CAPM$mjuH <- colMeans(X)
# Regression of historical returns on CAPM expected returns (in-sample)
check1 <- lm(mjuH ~ mju, data = CAPM)
# Extract out-sample returns
XOut <- XF[(nrow(X)+1):nrow(XF),]
# Historical returns (p.a.) (out-sample)
CAPM$mjuOut <- colMeans(XOut)
# Regression of historical returns on CAPM expected returns (out-sample)
check2 <- lm(mjuOut ~ mju, data = CAPM)
```

- **Results – In-Sample check:**

```
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 0.008766   0.003646   2.404   0.0231 *
mju         0.626182   0.422780   1.481   0.1497
```

- **Results – Out-Sample check:**

```
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.001693   0.005844  -0.290  0.77421
mju          2.078414   0.677682   3.067  0.00476 **
```

- **Note:** Generated with `summary(check1)` and `summary(check2)`

Note: No significant relationship between CAPM $\boldsymbol{\mu}$ and expected returns on the in-sample but surprisingly strong relationship on the out-sample (due to NVDA)

Check of the CAPM (2)

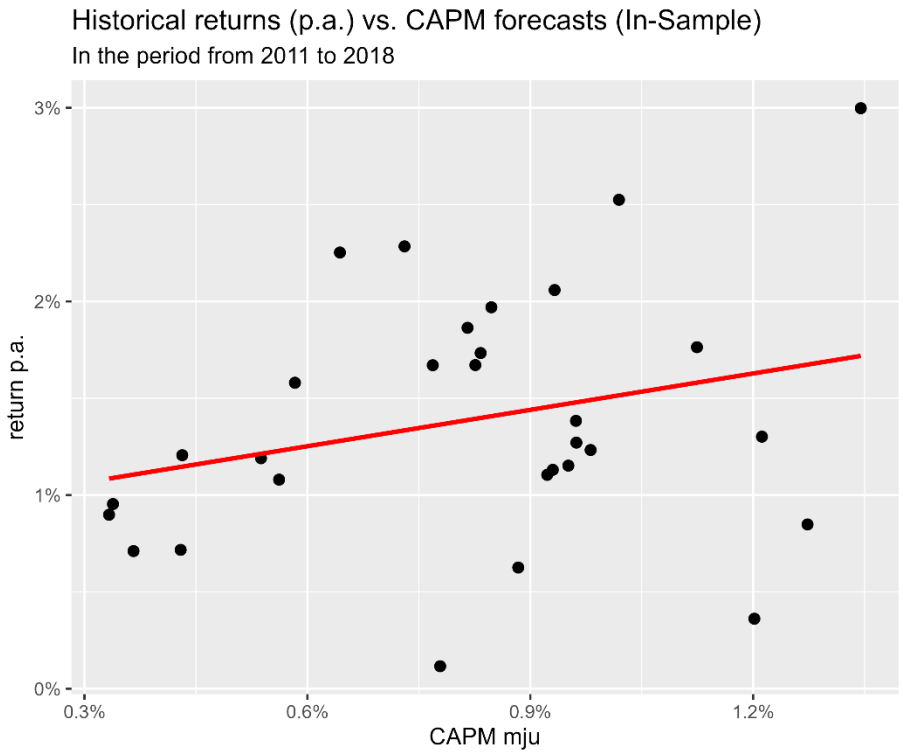
- **Plot the estimated regression on a chart (in-sample):**

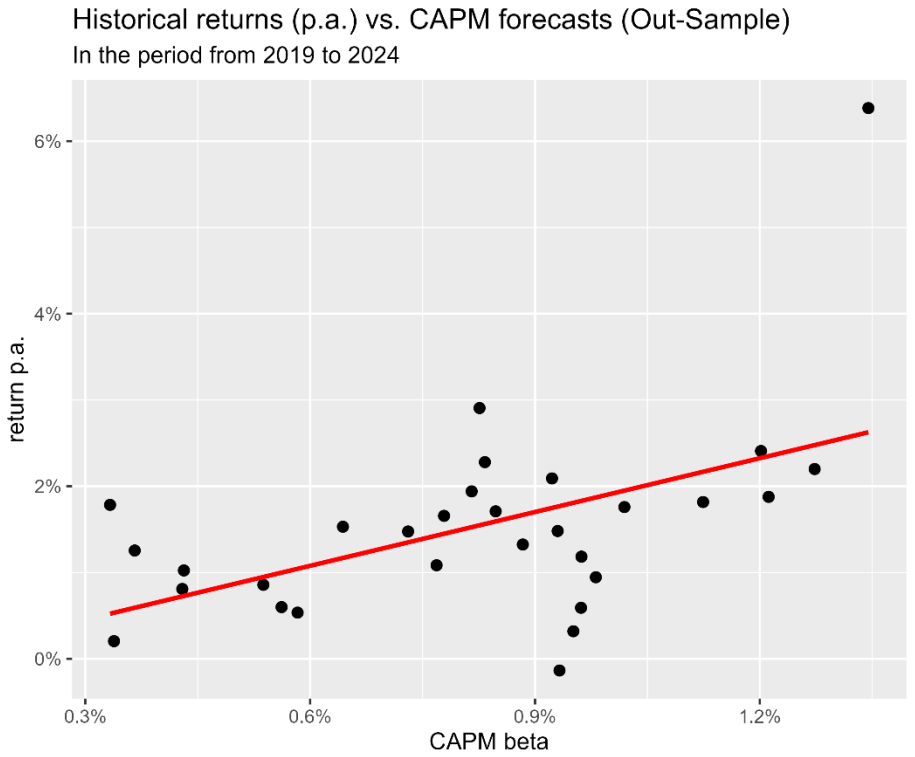
```
# Add regression forecasts to the CAPM table (in-sample)
CAPM$fit1 <- coef(check1)[1] + coef(check1)[2]*CAPM$mju
# Plot the relationship between historical returns and CAPM expected returns
c91 <- CAPM %>%
  ggplot() +
  geom_point(aes(x = mju, y = mjuH), size = 2) +
  geom_line(aes(x = mju, y = fit1), size = 1, color="red") +
  labs(title = "Historical returns (p.a.) vs. CAPM forecasts (In-Sample)",
       subtitle = paste0("In the period from ",year(tStart)," to ",sYear),
       x = "CAPM mju", y = "return p.a.") +
  scale_y_continuous(labels = scales::percent) +
  scale_x_continuous(labels = scales::percent)
```

- **Plot the estimated regression on a chart (out-sample):**

```
# Add regression forecasts to the CAPM table (out-sample)
CAPM$fit2 <- coef(check2)[1] + coef(check2)[2]*CAPM$mju
# Plot the relationship between historical returns and CAPM expected returns
c92 <- CAPM %>%
  ggplot() +
  geom_point(aes(x = mju, y = mjuOut), size = 2) +
  geom_line(aes(x = mju, y = fit2), size = 1, color="red") +
  labs(title = "Historical returns (p.a.) vs. CAPM forecasts (Out-Sample)",
       subtitle = paste0("In the period from ",sYear+1," to ",year(tEnd)),
       x = "CAPM beta", y = "return p.a.") +
  scale_y_continuous(labels = scales::percent) +
  scale_x_continuous(labels = scales::percent)
```

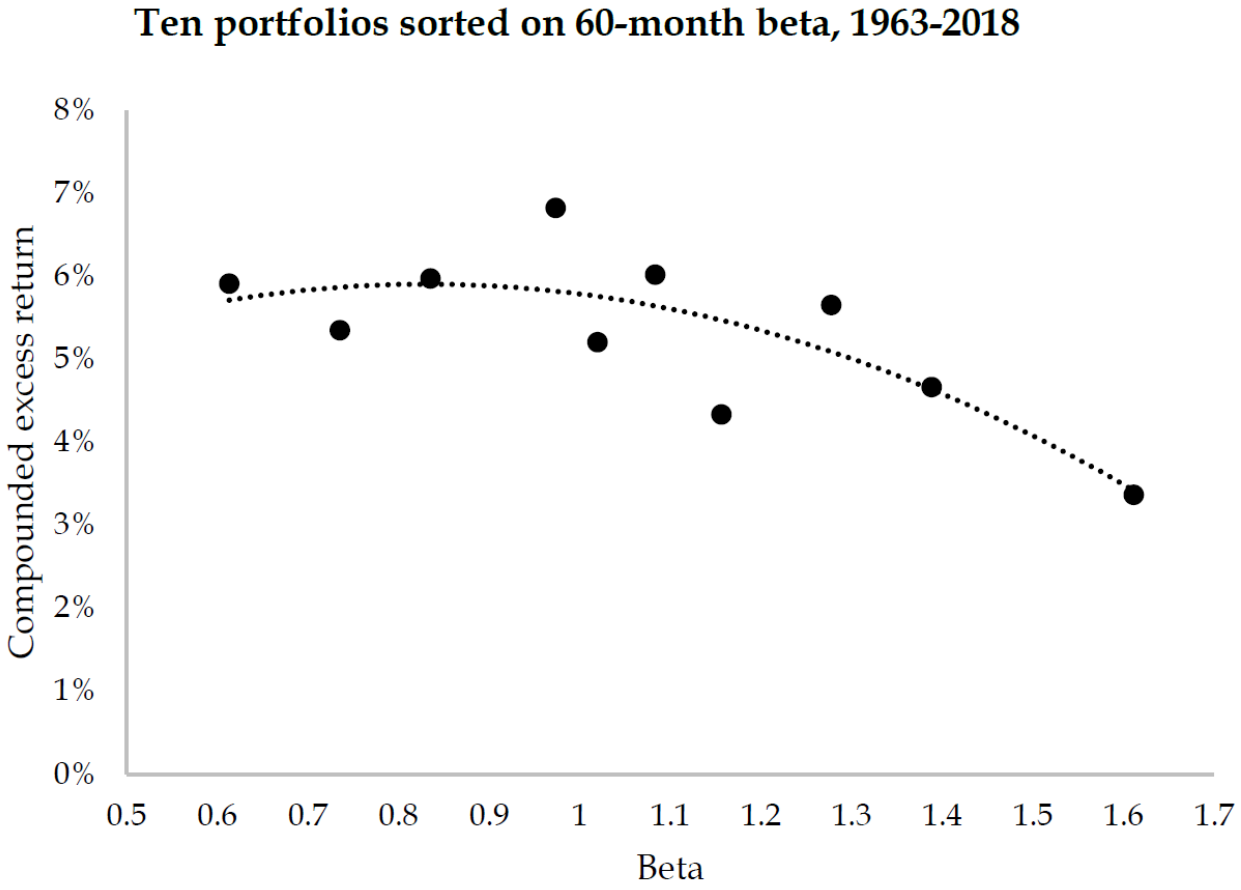
Check of the CAPM (3)





- **In-Sample:** Estimated relationship between CAPM expected returns and realized returns is weak (slope below 1) and insignificant
- **Out-Sample:** Strong relationship has been identified, but only due to one outlier observation (Nvidia), which has high beta and high realized returns (6% p.a.)
- **Note:** If the slope is lower than one, we can use the **Betting-Against-Beta** investment strategy
  → Short high-beta stocks and hedge the position with long low-beta stocks purchased on leverage so that the total portfolio beta is zero

Empirical testing of CAPM (1)

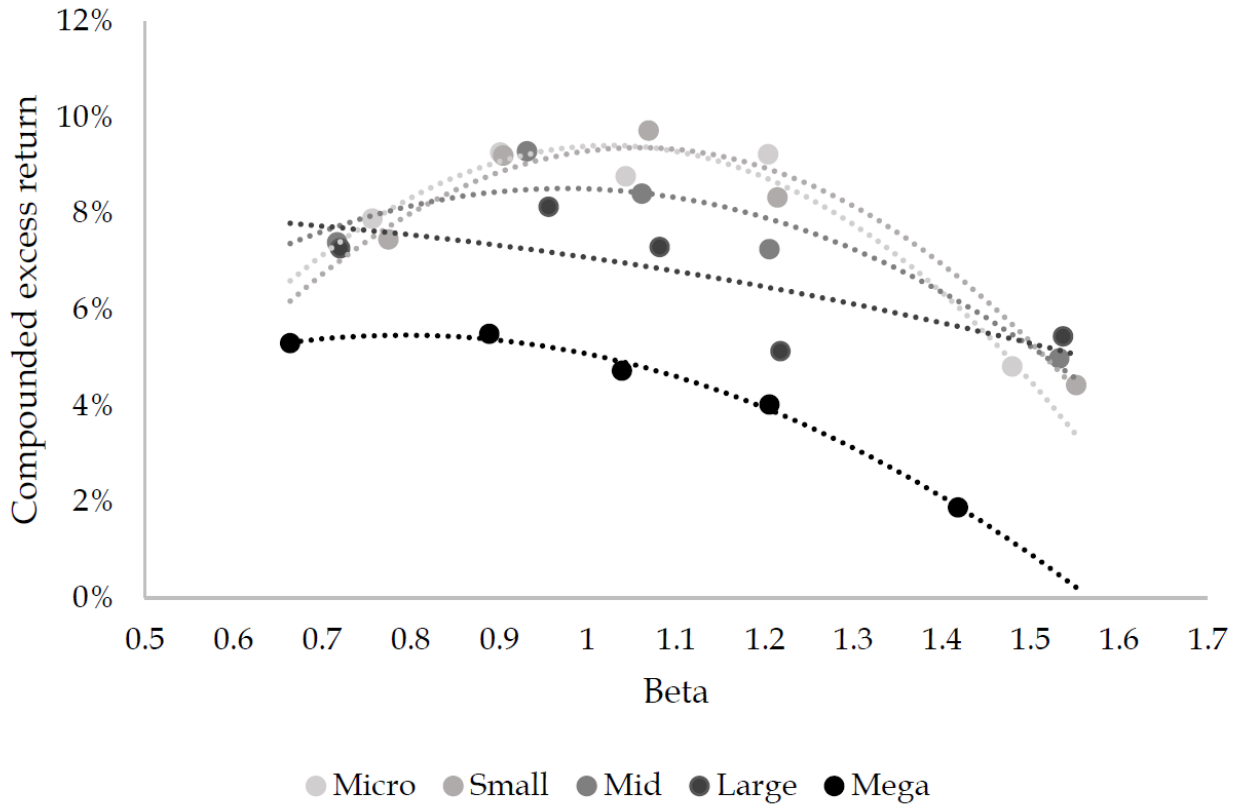


Source: Blitz, D., Vliet, P., Baltussen, G. (2019). The Volatility Effect Revisited

- Based on empirical testing CAPM does not seem to work well
  - Can partially be explained by the **effect of other factors**
  - Market risk premium is not the only risk premium

Empirical testing of CAPM (2)

5x5 portfolios sorted on size and 60-month beta, 1963-2018



Source: Blitz, D., Vliet, P., Baltussen, G. (2019). The Volatility Effect Revisited

- **CAPM** seems to work for some of the sub-groups with beta up to 1.1
- With additional factors the relationship becomes more linear, but is still too flat for low beta and high beta stocks – **volatility puzzle**